



الجمهورية العربية السورية
وزارة التعليم العالي والبحث العلمي
الجامعة الوطنية الخاصة
كلية الهندسة
قسم هندسة الحاسوب

حلقة بحث بعنوان

تحليل معمق للغة P4 وتأثيرها على البنية التحتية للخوادم والتطبيقات

An In-Depth Analysis of the P4 Language and its Impact on Server Infrastructure and Applications

إعداد الطالب

محمد خير الحوراني

إشراف

د. وسيم رمضان

العام الدراسي ٢٠٢٥ – ٢٠٢٦

شكر وتقدير

الملخص

شكّلت أجهزة الشبكات التقليدية ذات المنطق الثابت قيدياً بنويماً على تطور طبقة الشبكة لعقود متتالية، إذ حُجبت عن مطوّري البرمجيات أيّ قدرة على التأثير في آليات معالجة الحزم على مستوى الأجهزة. وقد جاءت لغة P4 (Programming Protocol-independent Packet Processors) لتُحدث قطيعة مع هذا النموذج، مُتّيحَةً للمطوّر تعريف سلوك مستوى البيانات برمّته — من تحليل الترويسات وصولاً إلى اتخاذ قرارات التوجيه — عبر كودٍ قابلٍ للتصريف والتشغيل على أجهزة حقيقية ومحاكيّات برمجية على حدٍّ سواء.

يتناول هذا البحث لغة P4 وتقنيات مستوى البيانات القابل للبرمجة من منظور مطوّر البرمجيات الشاملة، ساعياً إلى الإجابة عن سؤالٍ محوري: كيف يُعيد جعلُ الشبكة قابليّةً للبرمجة رسمَ حدود ما يستطيع المطوّر تحقيقه معمارياً وأداءً وأماناً في الأنظمة الحديثة؟ وتتبع الدراسة منهجيةً مزدوجة تجمع بين التحليل النظري لمكونات اللغة وبنيتها المعمارية، والتجريب العملي عبر بناء نموذجين أوليّين — جدارٍ نارٍ وموازن حمل — على نموذج المعالجة المرجعي V1Model، مع قياس نتائج حقيقية. ويستعرض البحث تطبيقات مستوى البيانات القابل للبرمجة في محاور تمسّ المطوّر مباشرة: تفريغ المهام الشبكية من الخوادم، والمراقبة عبر القياس الداخلي (INT)، وتعزيز الأمن على مستوى الأجهزة، ومرونة البروتوكولات. كما يُحلّل حالات تبنّي صناعية لدى شركات عالمية، ويرصد التحديات التقنية والتشغيلية، ويستشرف تقاطع P4 مع الذكاء الاصطناعي وشبكات الجيل السادس والحوسبة بلا خادم.

وتخلص الدراسة إلى أن لغة P4 لا تمثّل مجرد أداةٍ شبكية متخصصة، بل تُشكّل امتداداً طبيعياً لمنظومة أدوات مطوّر البرمجيات الشاملة، وأن القدرة على برمجة مستوى البيانات باتت كفاءةً تنافسية حقيقية في تصميم الأنظمة الموزعة عالية الأداء. الكلمات المفتاحية: لغة P4، مستوى البيانات القابل للبرمجة، الشبكات المُعرّفة بالبرمجيات، BMv2، تفريغ المهام الشبكية، موازنة الحمل، الجدار الناري، القياس الشبكي الداخلي، التطوير الشامل.

Abstract

Traditional fixed-function network devices have long imposed a structural constraint on the evolution of the network layer, denying software developers any meaningful influence over packet-processing logic at the hardware level. P4 (Programming Protocol-independent Packet Processors) represents a fundamental departure from this paradigm, enabling developers to define the complete behaviour of the data plane — from header parsing through forwarding decisions — in code that compiles and runs on both real programmable hardware and software simulators.

This research examines P4 and programmable data-plane technologies through the lens of the full-stack developer, addressing a central question: how does network programmability reshape what developers can achieve architecturally, in terms of performance, security, and system design? The study adopts a dual methodology that combines theoretical analysis of the language's components and architecture with practical experimentation through two prototypes — a stateless firewall and an L4 load balancer — built on the V1Model reference pipeline, with real measured results. It investigates programmable data-plane applications across domains directly relevant to full-stack development: server offloading of network functions, in-band network telemetry, hardware-level security enforcement, and custom protocol flexibility. It further analyses real-world deployments at major technology companies, evaluates the technical and operational challenges facing production adoption, and explores convergence with artificial intelligence, sixth-generation networking, and serverless computing.

The study concludes that P4 is not merely a specialized networking tool, but a natural extension of the full-stack developer's toolkit, and that programmable data-plane competency is becoming a genuine competitive advantage in the design of high-performance distributed systems.

Keywords: P4, programmable data plane, Software-Defined Networking, BMv2, network offloading, load balancing, firewall, in-band network telemetry, full-stack development.

جدول المحتويات

شكر وتقدير.....	2
الملخص.....	3
Abstract.....	4
جدول المحتويات.....	5
قائمة الأشكال والجداول.....	8
الفصل الأول: مدخل إلى الشبكات القابلة للبرمجة.....	9
١-١ الدافع والخلفية.....	9
٢-١ إشكالية البحث.....	9
٣-١ أسئلة البحث.....	9
٤-١ أهمية الموضوع وميزراته.....	10
٥-١ أهداف البحث.....	10
٦-١ نطاق البحث وحدوده.....	10
٧-١ منهجية البحث.....	11
٨-١ مفهوم الشبكات القابلة للبرمجة.....	11
٩-١ الخلفية النظرية: قصور الشبكات التقليدية.....	11
١-٩-١ تطوّر الشبكات الحاسوبية: لمحة تاريخية.....	11
٢-٩-١ بنية الشبكات التقليدية ومحدّداتها.....	11
٣-٩-١ الشبكات المُعرّفة بالبرمجيات وحدودها.....	12
٤-٩-١ المحفّزات التي مهّدت لـ P٤.....	12
١٠-١ الفجوة المعرفية في الأدبيات القائمة.....	12
١١-١ هيكل البحث.....	12
١٢-١ خلاصة الفصل.....	13
الفصل الثاني: لغة P٤ — المفاهيم والمعمارية.....	14
١-٢ مدخل إلى لغة P٤.....	14
٢-٢ بنية برنامج P٤: المكونات الأساسية.....	14
٣-٢ تعريف الترويسات وبنى البيانات.....	15
٤-٢ المُحلّل اللغوي: قراءة الحزمة.....	15
١-٤-٢ مثال تطبيقي: تعريف بروتوكول مخصّص.....	16
٥-٢ جداول المطابقة والإجراءات.....	16
٦-٢ المُعيد: إعادة تجميع الحزمة.....	17
٧-٢V1Model خطّ المعالجة الكامل: نموذج.....	17
وخصائصها الجوهرية P4٢-٨ قيود لغة.....	18
P4Runtime٢-٩ تكامل مستوى التحكم: واجهة.....	18

١٠-٢ خلاصة الفصل	19
الفصل الثالث: البنية التحتية والمنصات المستهدفة	20
١-٣ من الكود إلى الأجهزة	20
٢-٣ محاكي BMv2: بيئة المطور الأولى	20
٣-٣ شرائح Tofino: الأجهزة التجارية القابلة للبرمجة	20
٤-٣ بطاقات الشبكة الذكية (SmartNICs)	20
٥-٣ الدوائر المنطقية القابلة للبرمجة ميدانياً (FPGAs)	21
٦-٣ أهداف برمجية: eBPF و XDP	21
٧-٣ مقارنة شاملة بين الأهداف	21
٨-٣ سلسلة الأدوات وتجربة المطور (Toolchain & DevEx)	22
٩-٣ مرونة البروتوكولات: ترويسات مخصصة وبروتوكولات جديدة	22
١٠-٣ مسار التطوير الموصى به للمطور الشامل	22
١١-٣ خلاصة الفصل	23
الفصل الرابع: تطبيقات P٤ في الأداء والأمن والرقابة — والنموذج العملي	24
١-٤ مدخل	24
٢-٤ تفريغ المهام الشبكية (Offloading)	24
٣-٤ المراقبة والقياس الداخلي (In-band Network Telemetry)	24
٤-٤ التطبيقات الأمنية	25
٥-٤ النموذج العملي: تصميم التجربة وبيئتها	25
٦-٤ حالة الاستخدام الأولى: الجدار الناري عديم الحالة	26
٧-٤ حالة الاستخدام الثانية: موازن الحمل من المستوى الرابع	26
١-٧-٤ منهجية القياس	27
٨-٤ النتائج والتحليل	27
٩-٤ مناقشة ودلالات للمطور	28
١-٩-٤ حدود التجربة ومهددات الصدق	29
١٠-٤ خلاصة الفصل	29
الفصل الخامس: الواقع الصناعي، التحديات، والاستشراف	30
١-٥ التنبؤ الصناعي: دراسات حالة	30
١-١-٥ مايكروسوفت: تسريع شبكة Azure	30
٢-١-٥ ميتا وجوجل: التفريغ في مراكز البيانات	30
٣-١-٥ علي بابا والمنظومة الأوسع	30
٢-٥ تحديات المطور والقيود	30
٣-٥ التوجهات المستقبلية	31
٤-٥ الخاتمة	31

المراجع	33
الملحق: قوائم الكود الكاملة	35
ملحق أ: firewall.p٤ — الجدار الناري عديم الحالة	35
ملحق ب: load_balancer.p٤ — موازن الحمل من المستوى الرابع	36
ملحق ج: ملاحظة حول إعادة الإنتاج	37

قائمة الأشكال والجداول

الأشكال

الشكل ١-٢: خط معالجة الحزمة وفق نموذج V1Model

الشكل ٢-٢: الفصل بين مستوى التحكم ومستوى البيانات عبر P4Runtime

الشكل ١-٤: طبولوجيا تجربة النموذج العملي

الشكل ٢-٤: نتيجة الجدار الناري — دقة تصنيف كاملة

الشكل ٣-٤: توزيع حركة موازن الحمل على الخوادم

الجداول

جدول ١-١: مقارنة بين النماذج الثلاثة لإدارة الشبكة

جدول ١-٢: المكونات البنيوية الخمسة لبرنامج P4

جدول ١-٣: مقارنة بين أهداف تشغيل برامج P4

جدول ١-٤: ملخص النتائج المقيسة للنموذجين العمليين

جدول ٢-٤: أين تُنفَّذ الوظيفة — التطبيق مقابل مستوى البيانات

الفصل الأول: مدخل إلى الشبكات القابلة للبرمجة

١-١ الدافع والخلفية

شهد العقدان الأولان من الألفية الثالثة تحولاً جذرياً في طبيعة الشبكات الحاسوبية وأنماط الطلب عليها؛ إذ لم تعد الشبكة مجرد قناة لنقل البيانات بين طرفين، بل غدت البنية التحتية الحاملة لاقتصادٍ رقميٍّ منتشعب يضمّ خدمات البثّ المرئي، والحوسبة السحابية، وتطبيقات الذكاء الاصطناعي الموزعة، وإنترنت الأشياء. وقد أفرز هذا التحول ضغطاً متصاعداً على أجهزة الشبكات التقليدية التي صُممت في حقبة مختلفة، وبُنيت على فرضية مفادها أنّ بروتوكولات الاتصال ثابتة ومتوقعة لا تتبدّل.

تقوم الشبكات التقليدية على مبدأ الفصل الصارم بين مستوى التحكم (Control Plane) ومستوى البيانات (Data Plane)؛ غير أنّ مستوى البيانات ظلّ تاريخياً محجوباً خلف دوائر إلكترونية متخصصة لا يملك المطور أيّ سلطةٍ عليها. فالمحول أو الموجه التقليدي يُحلّل الحزمة وفق منطقي صارمٍ محدّد سلفاً من قِبل المصنّع، ولا يتيح أيّ إمكانيةٍ لتعريف بروتوكولاتٍ جديدة أو تعديل سلوك المعالجة على مستوى الأجهزة [٢٣]. وقد شكّل هذا عائقاً أمام الابتكار في طبقة الشبكة لعقودٍ متتالية.

في هذا السياق ظهرت لغة P4 — اختصاراً لـ Programming Protocol-independent Packet Processors — عام ٢٠١٤ استجابةً مباشرة لهذه الإشكالية [١]. فلأوّل مرّة في تاريخ الشبكات، أصبح بمقدور المطور أن يُعرّف بنفسه كيف تُحلّل أيّ ترويسة، وكيف تُعالج الحزمة، وأي إجراء يُتخذ بحقّها — كلّ ذلك في كودٍ يُصرّف ويُحلّل على الأجهزة الحقيقية أو على المحاكيات البرمجية. وقد فتحت هذه اللغة الباب أمام نموذجٍ جديد من الشبكات القابلة للبرمجة يُعيد رسم حدود ما هو ممكنٌ في طبقة الشبكة.

ومن موقع مطوّر البرمجيات الشاملة (Full-Stack Developer) تحديداً، يكتسب هذا التحول دلالةً خاصةً: فهذا المطور يتحكّم بكلّ طبقةٍ في النظام الذي يبنيه — من قاعدة البيانات إلى منطق الخادم إلى واجهة المستخدم — إلّا طبقة الشبكة، التي طالما تعامل معها بوصفها صندوقاً أسود يُدار ولا يُبرمج. تقترح هذه الدراسة أنّ P4 تفتح هذا الصندوق، وتحوّل الشبكة من قيدٍ مفروض إلى مكوّنٍ برمجيّ قابلٍ للتشكيل ضمن منظومة أدوات المطور.

٢-١ إشكالية البحث

رغم أنّ لغة P4 وتقنيات مستوى البيانات القابل للبرمجة قد استقطبت اهتماماً واسعاً في أبحاث الشبكات وهندسة الاتصالات، إلّا أنّ أثرها على مجال التطوير الشامل لا يزال يفتقر إلى دراسةٍ منهجيةٍ كافية، ولا سيّما في الأدبيات العربية. وتنطلق هذه الإشكالية من ملاحظةٍ جوهرية: إنّ طبقة الشبكة ظلّت بالنسبة للمطور صندوقاً أسود يعلم بوجوده لكنّه لا يستطيع التأثير في آليّاته الداخلية، فترسّخ في تصميم الأنظمة نمطٌ يتقبّل ضمناً حدوداً ما تستطيع الشبكة تقديمه بدلاً من توظيفها كمكوّنٍ قابلٍ للبرمجة. وتزداد هذه الإشكالية حدّةً مع النموّ الهائل في حجم البيانات المتدفقة، حيث باتت قرارات معالجة الحزم — التي كانت من اختصاص مهندسي الشبكات حصراً — ذات أثرٍ مباشر في أداء التطبيقات وكلفة تشغيلها.

٣-١ أسئلة البحث

يسعى البحث إلى الإجابة عن ثلاثة أسئلةٍ رئيسية:

١. أولاً: ما الذي تغيّر فعلياً في القيود التي كانت مفروضة على المطورين بسبب ثبات مستوى البيانات في الشبكات التقليدية، وكيف تُخفّفها لغة P4؟

٢. **ثانياً:** كيف تنعكس قدرات مستوى البيانات القابل للبرمجة على القرارات المعمارية للتطبيقات الحديثة، من حيث تفريغ المهام الشبكية، والبروتوكولات المخصصة، وتحليل الأداء في الزمن الحقيقي؟
٣. **ثالثاً:** ما الفجوة القائمة بين أدوات مطور البرمجيات الشاملة ومهاراته اليوم وبين متطلبات بناء أنظمة تستفيد من قابلية برمجة الشبكة، وكيف يمكن سدّها؟

١-٤ أهمية الموضوع ومبرراته

تتجلى أهمية هذا البحث من زوايا متكاملة. **من الناحية التقنية**، يمثل مستوى البيانات القابل للبرمجة تحولاً نوعياً في القدرة التعبيرية المتاحة للمطور على طبقة الشبكة؛ فبدلاً من انتظار المصنّعين لإضافة دعم لبروتوكول جديد، بات بمقدور الفريق الهندسي تعريف ذلك البروتوكول بنفسه وتشغيله مباشرةً على أجهزة قابلة للبرمجة، في ما يُشبه ما فعلته لغات البرمجة العالية المستوى تاريخياً مع طبقة المعالجات.

ومن الناحية الاقتصادية، وثّقت دراسات كبرى شركات التكنولوجيا — كميّنا وجوجل وعلي بابا ومايكروسوفت — أنّ نقل جزء من منطق التطبيق إلى مستوى الشبكة، كموازنة الحمل والتخزين المؤقت وفلتر الحزم، يُخفّض ملحوظاً استهلاك موارد الخوادم ويُقلّص زمن الاستجابة في بيئات الإنتاج [٧][١٣]، وهو ما يُترجم إلى وفورات تشغيلية حقيقية. **ومن الناحية الأكاديمية**، يسدّ البحث فجوة في الأدبيات المتخصصة التي تقارب P4 غالباً من منظور هندسة الشبكات البحث، بينما يتبنّى هذا البحث زاوية مطور البرمجيات الشاملة التي لم تحظ بعناية كافية، خصوصاً عربياً.

١-٥ أهداف البحث

- **بناء الفهم المفاهيمي:** تقديم عرضٍ تأسيسيٍّ وافٍ لمفهوم مستوى البيانات القابل للبرمجة ولغة P4 ومكوّناتها وبنيتها المعمارية، بأسلوبٍ يُخاطب المطور الشامل أكثر مما يخاطب متخصص الشبكات.
- **تحليل الأثر التقني:** دراسة كيفية تأثير قابلية برمجة مستوى البيانات في منظومة التطوير الشامل، من تصميم البروتوكولات وتفرغ المهام إلى الأمن والقياس في الزمن الحقيقي.
- **استعراض الواقع الصناعي:** تحليل حالات استخدامٍ حقيقية لدى شركات رائدة، وتقييم نضج أدوات P4 وجاهزيتها للإنتاج.
- **التجريب العملي:** بناء نموذجين أوليين — جدار ناري وموازن حمل — والتحقّق منهما بقياساتٍ حقيقية على نموذج المعالجة المرجعي V1Model.
- **استشراف المستقبل:** رسم ملامح تقاطع P4 مع الذكاء الاصطناعي وشبكات الجيل السادس والحوسبة على الحافة.

١-٦ نطاق البحث وحدوده

يُعنى البحث بدراسة لغة P4 من منظور المطور الشامل، ويشمل نطاقه: اللغة في إصدارها P4₁₆ المعتمد حالياً، ومحامي البرمجيات BMv2 ونموذج V1Model بوصفهما البيئة التجريبية الرئيسة؛ وواجهة P4Runtime بوصفها بروتوكول التواصل بين مستويي التحكم والبيانات؛ وتأثيرات قابلية البرمجة على طبقة التطبيق وتصميم الأنظمة الموزعة. وفي المقابل يستبعد البحث من نطاقه الأساسي التفاصيل الدقيقة للتنفيذ الإلكتروني على مستوى الدوائر المدمجة، والمقارنات الكميّة التفصيلية بين منصّات العتاد التجارية (لما تتطلب من بيانات قياسٍ متخصصة لم تكن متاحة في إطار هذا البحث)، وبروتوكولات الشبكات اللاسلكية، إذ ينصبّ التركيز على الشبكات السلكية في بيئات مراكز البيانات وخدمات الإنترنت.

٧-١ منهجية البحث

اعتمد البحث منهجية مزدوجة تجمع بين المنهج الوصفي التحليلي والمنهج التجريبي؛ يُوظف الأول في استعراض المفاهيم ومراجعة الأدبيات وتحليل حالات الاستخدام الصناعية، ويُوظف الثاني في بناء النموذجين الأوليين والتحقق منهما. وتستند المادة العلمية إلى مصادر أولية تشمل المواصفة الرسمية للغة P4 [٣]، وأوراق المؤتمرات المُحكّمة في SIGCOMM و NSDI [١][٢][٥][٩]، إضافةً إلى تقارير الشركات التقنية الكبرى ووثائق المشاريع مفتوحة المصدر [١٧][١٨][١٩]. وفي الجانب التجريبي، تُمدّجت دلالات نموذج المعالجة V1Model في بيئة برمجية، وسيقت إليها حزمٌ حقيقية مُصاغة بأداة scapy لقياس صحة المنطق وسلوكه كمياً، على النحو المفصّل في الفصل الرابع.

٨-١ مفهوم الشبكات القابلة للبرمجة

يقوم مفهوم الشبكات القابلة للبرمجة على فكرة بسيطة في صياغتها عميقة في أثرها: أن يكون سلوك معالجة الحزمة نفسه — لا مجرّد القواعد التي تُملأ بها الجداول — قابلاً للتعريف برمجياً من خارج الجهاز. وهذا يتجاوز ما قدّمته الشبكات المُعرّفة بالبرمجيات (SDN) التي حرّرت مستوى التحكّم وأبقت مستوى البيانات على حاله. ففي الشبكة القابلة للبرمجة بلغة P4، يكتب المطوّر برنامجاً يُحدّد: أيّ الترويسات تُقرأ ومن أين، وكيف تُطابَق حقولها، وأي إجراء يُنفَّذ، وكيف تُعاد الحزمة إلى السلك — ثم يُصرّف هذا البرنامج إلى الهدف المطلوب. بهذا تنتقل الشبكة من كونها بنية جامدة تُشتري بمواصفات ثابتة إلى كونها منصّة برمجية تتطوّر بوتيرة البرمجيات لا بوتيرة العتاد.

٩-١ الخلفية النظرية: قصور الشبكات التقليدية

٩-١-١ تطوّر الشبكات الحاسوبية: لمحة تاريخية

نشأت الشبكات الحاسوبية الحديثة في رحم الحاجة العسكرية والأكاديمية في ستينيات القرن الماضي، حين صُمّمت شبكة ARPANET لتوفير اتصالٍ مرِنٍ يصمد أمام الأعطال الجزئية. وقامت هذه الشبكات منذ نشأتها على مبدأ توزيع الأدوار: تتولّى الأجهزة الشبكية — من موجّهاتٍ ومحوّلات — اتخاذ قرارات التوجيه، بينما تتصرف الأجهزة الطرفية إلى تشغيل التطبيقات. وأفضى هذا إلى البنية ثنائية المستوى التي باتت جوهر تصميم الشبكات: مستوى البيانات المعنيّ بمعالجة الحزم وإعادة توجيهها بسرعةٍ عالية، ومستوى التحكّم المعنيّ باتخاذ القرارات وتحديث جداول التوجيه. وعلى امتداد العقود التالية تطوّرت البروتوكولات تطوّراً متدرّجاً لكتّنها ظلّت محكومةً بمنطق التوحيد القياسي؛ فبروتوكول IPv2 الذي أُطلق عام ١٩٨١ [١٤] لا يزال يحمل نسبةً ضخمة من حركة الإنترنت، و Ethernet المُعرّف في السبعينيات [١٥] لا يزال يهيمن على الشبكات المحلية. وهذه الاستمرارية ليست دليل كمالٍ بل دليلٌ على باهظ تكاليف التغيير في بنية ترسّخت لعقود، وتُجسّد ما يُعرف بـ «جمود الموروث المعماري».

٩-١-٢ بنية الشبكات التقليدية ومحدّداتها

في الموجه التقليدي يعمل مستوى التحكّم على معالج عام يُشغّل بروتوكولات توجيه ك OSPF و BGP، فيبني جداول التوجيه ويحدّثها استجابةً لتغيّرات الطوبولوجيا. أمّا مستوى البيانات فيعمل على دوائر إلكترونية متخصصة فائقة السرعة تُعرف بـ ASIC، مهمتها الوحيدة البحث عن وجهة الحزمة الواردة في جدول التوجيه وفق خوارزمية «أطول تطابق بادئة» (LPM) ثم إعادة إرسالها عبر المنفذ المناسب في أقصر وقت، مع تحديث حقول كقيمة TTL. وهذه الدوائر قادرةٌ على معالجة عشرات الملايين من الحزم في الثانية، لكنّ ثمن هذا الأداء هو الجمود المطلق: فالـ ASIC يُصنّع لوظيفةٍ واحدة لا تتبدّل، ولا يمكن تعديل منطقته بعد التصنيع [٦]. فإذا أراد المهندس إضافة دعمٍ لبروتوكولٍ جديد وجد نفسه أمام خيارٍ وحيد: استبدال الجهاز

بأكمله. يُضاف إلى ذلك أنّ واجهات الإدارة التقليدية — من SNMP و CLI إلى NETCONF/YANG — تبقى ضمن حدود ما يسمح به المصنّع؛ فالمشغل لا يرى من الجهاز إلا ما أراد المصنّع إظهاره، ولا يُعدّل إلا ما أتاح تعديله، فتغدو العلاقة علاقةً مستخدم بتطبيق مغلق المصدر لا علاقة مطوّر ببنية قابلة للبرمجة.

٣-٩-١ الشبكات المُعرّفة بالبرمجيات وحدودها

في مطلع الألفية برز مفهوم الشبكات المُعرّفة بالبرمجيات (SDN) استجابةً لهذه القيود، قائماً على فكرةٍ جوهرية: الفصل الكامل بين مستويي التحكم والبيانات، وإخراج الأول من الجهاز إلى برنامجٍ مركزيّ يُسمّى المتحكّم (Controller) يرى الشبكة كوحدةٍ واحدة [٢٣]. وانتشر بروتوكول OpenFlow بوصفه اللغة القياسية للتواصل بين المتحكّم والأجهزة [٢]، مُتيحاً تثبيت قواعد التوجيه وتعديلها ديناميكياً، وهو ما مثّل قفزةً في مرونة الإدارة لا سيّما في مراكز البيانات. غير أنّ SDN ترك مستوى البيانات على حاله؛ فـ OpenFlow يُعدّل القواعد داخل جداولٍ بنيّتها ثابتة، ولا يملك تأثيراً في آلية تحليل الترويسات ولا في منطق المعالجة. وبعبارةٍ مكثّفة: بمنحك SDN مفتاح إضاءةٍ أكثر مرونة، لكنّ المصباح نفسه — منطق معالجة الحزم — يبقى محدّداً سلفاً ومثبّثاً في الجهاز. كما عانى OpenFlow من «تضخّم الميزات» [١]: فكلّما احتاج المطوِّرون دعم حقْلٍ أو بروتوكولٍ جديدٍ لزمهم تعديل المواصفة القياسية وانتظار تبني المصنّعين، في مسارٍ قد يمتدّ سنوات. وقد كشف هذا عن الحاجة إلى نموذجٍ مختلفٍ جذرياً لا يكفي ببرمجة مستوى التحكم بل يُبرمج مستوى البيانات ذاته.

جدول ١-١: مقارنة بين النماذج الثلاثة لإدارة الشبكة

النموذج	مستوى التحكم	مستوى البيانات
الشبكات التقليدية	موزّع داخل الأجهزة	ثابت في الجهاز (ASIC)
الشبكات المُعرّفة بالبرمجيات (SDN)	مركزي (Controller)	ثابت — جداولٍ بنيّتها محدّدة سلفاً
الشبكات القابلة للبرمجة (P4)	مركزي عبر P4Runtime	قابل للبرمجة بالكامل عبر P4

٤-٩-١ المحفّزات التي مهّدت لـ P4

لم يكن الطريق إلى P4 ممكناً قبل نضج تقنيتين: الدوائر المنطقية القابلة للبرمجة ميدانياً (FPGA) وبطاقات الشبكة الذكية (SmartNIC)، اللتين أتاحتا لأوّل مرّة بيانات أجهزةٍ قابلة للبرمجة بعد التصنيع مع أداءٍ يقترب من الـ ASIC. وعلى الصعيد الذهني، أدّى OpenFlow دوراً تأسيسياً: فقد أثبت أنّ مجتمع الشبكات مستعدٌّ لتقبّل برمجة الشبكة بشكلٍ مجرّد، وأنّ الأجهزة يمكنها تلقّي تعليماتها من خارجها. وفي عام ٢٠١٤ نشر باحثون من ستانفورد وبرينستون وغيرهما الورقة التأسيسية «P4: [١] Programming Protocol-Independent Packet Processors»، مُرسّخين ثلاثة مبادئ صارت ركائز اللغة: الاستقلالية عن البروتوكول، والاستقلالية عن الهدف، وقابلية إعادة الضبط في وقت التشغيل.

١٠-١ الفجوة المعرفية في الأدبيات القائمة

يكشف استعراض الأدبيات توجّهاً غالباً يقارب P4 من منظور هندسة الشبكات أو تصميم الأجهزة، بينما تبقى المقاربة من زاوية مطوّر البرمجيات الشاملة — المعنيّ بالأثر المعماري على التطبيقات، والتكامل مع منظومة أدواته، والجدوى العملية في مشاريع الإنتاج — شحيحةً، ولا سيّما عربياً. يتعمّد هذا البحث تبني تلك الزاوية في تناول تقنية نشأت في رحم أبحاث الشبكات، مُقدِّماً إياها في سياقٍ يُخاطب الشواغل المهنية لهذه الفئة من المطوِّرين.

١١-١ هيكل البحث

يتوزّع هذا البحث على خمسة فصول. يُرسي **الفصل الأول** الأساس المفاهيمي ويستعرض قصور الشبكات التقليدية الذي دفع نحو القابلية للبرمجة. ويُقدّم **الفصل الثاني** لغة P4 بعمقٍ تقنيٍّ كافٍ — من الترويسات والمُحلّ إلى المطابقة والإجراء وخطّ المعالجة الكامل وتكامل مستوى التحكم. ويتناول **الفصل الثالث** البنية التحتية والمنصّات المستهدفة وسلسلة الأدوات ومرونة البروتوكولات. ويُشكّل **الفصل الرابع** جوهر المساهمة التطبيقية، إذ يُحلّل تطبيقات P4 في الأداء والأمن والرقابة، ويعرض النموذجين العمليّين ونتائجهما المقيسة. ويختتم **الفصل الخامس** بالواقع الصناعي والتحديات والاستشراف والخاتمة.

١٢-١ خلاصة الفصل

يُتّضح مما سبق أنّ قيود الشبكات التقليدية لم تكن عوائق عرضية تُحلّ بتحديثٍ برمجيّ، بل قيوداً بنبوية مُضمّنة في فلسفة التصميم: فالـ ASIC يفعل ما صُنِعَ له بإتقانٍ مذهل، لكنّه لا يفعل شيئاً سواه. وقد خلق هذا هوّةً حقيقية بين ما تحتاجه الأنظمة الحديثة من الشبكة وما تستطيع الشبكة تقديمه. جاءت P4 — بعد أن مهّدت لها تجربة SDN و OpenFlow — لتقدّم إجابةً جذرية: لا تكفي بمنح المطوّر سلطةً على مستوى التحكم، بل تمنحه القدرة التعبيرية الكاملة على مستوى البيانات ذاته، وهو ما يفصّله الفصل التالي.

الفصل الثاني: لغة P4 — المفاهيم والمعمارية

١-٢ مدخل إلى لغة P4

تتنتمي لغة P4 إلى فئة اللغات المجالية المتخصصة (Domain-Specific Languages)، غير أنها تختلف اختلافاً جوهرياً عن نظائرها الشائعة كـ SQL وRegex؛ إذ لا تعمل على بياناتٍ مخزنة أو نصوص، بل على تدفقات حزم حية تعبر الشبكة بسرعاتٍ تقاس بالغيغابت في الثانية. وقد صيغت منذ تأسيسها على ثلاثة مبادئ حاكمة [١][٣]: الاستقلالية عن البروتوكول، إذ لا تفترض اللغة أي معرفة مسبقة ببروتوكولات الاتصال — فهي لا «تعرف» Ethernet ولا IPv4 ولا TCP، بل يُعرّفها المطور من الصفر كما يُعرّف بنية بيانات في لغة C؛ والاستقلالية عن الهدف، إذ يتكفل المصنّف بترجمة البرنامج نفسه إلى تمثيلٍ قابلٍ للتنشغيل على محاكٍ كـ BMv2 أو شريحة FPGA أو جهازٍ تجاري كـ Tofino؛ وقابلية إعادة الضبط في وقت التشغيل، إذ يمكن تعديل سلوك الجهاز أثناء عمله عبر واجهة P4Runtime [٩١]. ويتبنّى هذا الفصل نهجاً تصاعدياً: من البنى الأساسية للترويسات، إلى المُحلّل، فالمطابقة والإجراء، وصولاً إلى خطّ المعالجة الكامل ثمّ تكامل مستوى التحكم.

٢-٢ بنية برنامج P4: المكونات الأساسية

يتشكّل أيّ برنامج P4 من خمسة مكونات بنوية تعمل في تناسق لتشكيل خطّ معالجة الحزمة. ويمكن استيعاب العلاقة بينها بتشبيه الحزمة بطردٍ بريدي: فالمُحلّل (Parser) يقرأ الملتصق ويستخرج معلومات المُرسِل والمستقبل، وجداول المطابقة (Match-Action Tables) تبحث في دليل العناوين وتحدّد ما يجب فعله، والإجراءات (Actions) تُنفّذ الفعل من إعادة توجيه أو تعديل أو حذف، والمُعيد (Deparser) يُعيد ختم الطرد قبل إرساله، وكتلة التحكم (Control) تُنسّق التسلسل بين هذه العمليات. ويُجسّد الجدول التالي هذه المكونات الخمسة ووظيفة كلّ منها.

جدول ١-٢: المكونات البنوية الخمسة لبرنامج P4 ووظائفها

المكوّن	الوظيفة
المُحلّل (Parser)	استخراج الترويسات من بايتات الحزمة وفق آلة حالاتٍ متناهية
جداول المطابقة (Tables)	مطابقة حقول الترويسات مع قواعد الجداول لتحديد الإجراء
الإجراءات (Actions)	تعديل الترويسات والبيانات الوصفية وضبط منفذ الخروج
المُعيد (Deparser)	إعادة تسلسل الترويسات المعدلة إلى تدفق بايتات متّصل
كتلة التحكم (Control)	تنسيق ترتيب تنفيذ الجداول وتوجيه تدفق المعالجة

ولتثبيت الصورة الكليّة قبل الغوص في التفاصيل، يُظهر الهيكل العظمي التالي كيف تتجاور المكونات الخمسة في برنامجٍ مكتمل وفق نموذج V1Model؛ وهو القلب الذي يبني عليه النموذجان العمليّان في الفصل الرابع:

```
#include <core.p4>
#include <v1model.p4>

struct headers { /* الترويسات */ }
struct metadata { /* بيانات وصفية مؤقتة */ }

parser MyParser(...) { /* قراءة الحزمة */ }
control MyVerifyChecksum(...) { /* تحقّق */ }
control MyIngress(...) { /* المطابقة والإجراء */ }
```

```
control MyEgress(...) { /* معالجة الخروج */ }
control MyComputeChecksum(...) { /* إعادة حساب */ }
control MyDeparser(...) { /* إعادة الكتابة */ }
```

```
V1Switch(MyParser(), MyVerifyChecksum(), MyIngress(),
        MyEgress(), MyComputeChecksum(), MyDeparser()) main;
```

يُجسّد السطر الأخير نقطة التجميع: إذ تُمرّر المكونات السّنة إلى حزمة V1Switcher بالترتيب الذي يفرضه نموذج المعالجة، فينشأ من تجاوزها خطّ معالجة مكتمل قابل للتصريف والتشغيل.

٣-٢ تعريف الترويسات وبنى البيانات

الترويسة في P4 تعريف صريح لتسلسل من حقول البت محدّدة الحجم تُشكّل بروتوكولاً معيّناً. وخلافاً للغات العامة التي تتعامل مع البيانات بوحدات البايت أو الكلمة، تتعامل P4 على مستوى البت الواحد، لأنّ ترويسات الشبكة مُعرّفة بهذه الدقّة في مواصفات البروتوكولات. ويوضّح المثال التالي تعريف ترويسة Ethernet وترويسة IPv4 كما في نموذجنا العملي:

```
header ethernet_t {
    bit<48> dstAddr;    // عنوان الوجهة (MAC)
    bit<48> srcAddr;    // عنوان المصدر (MAC)
    bit<16> etherType;  // نوع الحمولة (0x0800 = IPv4)
}
header ipv4_t {
    bit<4> version; bit<4> ihl; bit<8> diffserv;
    bit<16> totalLen; bit<16> id; bit<3> flags; bit<13> fragOffset;
    bit<8> ttl; bit<8> proto; bit<16> hdrChecksum;
    bit<32> srcAddr; bit<32> dstAddr;
}
```

إلى جانب الترويسات المُستخرجة من بايتات الحزمة، تدعم P4 البيانات الوصفية (Metadata): حقول لا وجود لها في الحزمة الفعلية، يُنشئها المطوّر لحمل حالة مؤقتة تُستخدم في اتّخاذ القرار ثم تُهمل قبل الإرسال — كأن نحفظ فيها رقم الخادم الخلفي الذي اختاره موازن الحمل. ويُجسّد هذا النوع الفرق الجوهرى بين P4 وبرمجة الشبكات التقليدية: فالمطوّر لا يقتصر على قراءة ما تحمله الحزمة، بل يُنشئ حالة خاصّة به يحملها عبر مراحل المعالجة. وتُجمّع الترويسات والبيانات الوصفية في بنى مجمّعة (Structs) تُمرّر بين مكونات البرنامج كوحدة واحدة.

٤-٢ المُحلّل اللغوي: قراءة الحزمة

المُحلّل في P4 آلة حالاتٍ متناهية (Finite State Machine) يُعرّفها المطوّر صراحةً [٣]. تبدأ من حالة ابتدائية ثابتة تُسمّى start، وتنتهي إمّا في accept حين يكتمل التحليل، أو في reject حين تُنتهك قواعده، وتنتقل بينهما عبر حالاتٍ وسيطة تُعبّر كلّ منها عن مرحلة في قراءة الحزمة. وهذا التصميم ليس اعتباطياً؛ فالآلة الحالات تُترجم بكفاءة عالية إلى دوائر إلكترونية، مما يجعل التحليل قابلاً للتشغيل بسرعة الأسلاك. ويُظهر المثال التالي مُحلّلاً ينتقل من Ethernet إلى IPv4 إلى TCP وفق قيمة الحقل التالي — وهو ما يُسمّى التحليل الانتقائي:

```
parser MyParser(packet_in pkt, out headers hdr, ...) {
    state start { transition parse_ethernet; }
    state parse_ethernet {
        pkt.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType) {
            0x0800: parse_ipv4; // إن كانت IPv4 تابع التحليل
            default: accept;    // وإلا قبل الحزمة كما هي
        }
    }
}
```

```

    }
    state parse_ipv4 {
        pkt.extract(hdr.ipv4);
        transition select(hdr.ipv4.proto) { 6: parse_tcp; default: accept; }
    }
    state parse_tcp { pkt.extract(hdr.tcp); transition accept; }
}

```

تتجلى قوة المُحلّل حين نقارنه بنظيره في الأجهزة التقليدية: فبدلاً من تحليل كلّ ترويسة في كلّ حزمة دون تمييز، يُعرّف المطوّر مساراتٍ مختلفة لأنواع حزمٍ مختلفة، ويستطيع إضافة دعمٍ لبروتوكولاتٍ مخصصة لم يسبق تعريفها في أيّ معيار — وهذه القدرة تحديداً هي ما يُميّز P4 عمّا سبقها.

٢-٤-١ مثال تطبيقي: تعريف بروتوكولٍ مخصص

لتجسيد مبدأ الاستقلالية عن البروتوكول عملياً، لنفترض أننا نريد تعريف ترويسةٍ مخصصةٍ بالكامل — لنُسمّيها myTunnel — تحمل مُعرّفاً للوجهة المنطقية داخل بنيةٍ متعدّدة المستأجرين. لا يوجد لهذه الترويسة أيّ معيارٍ عالمي؛ نحن من نخترعها، ونُعرّفها في P4 كما نُعرّف أيّ بنية، ثمّ نُعلّم المُحلّل كيف يتعرّف عليها انطلاقاً من قيمةٍ خاصة في حقل etherType:

```

header myTunnel_t {
    bit<16> proto_id;    // نوع الحمولة الداخلية
    bit<16> dst_id;      // مُعرّف الوجهة المنطقية
}

state parse_ethernet {
    pkt.extract(hdr.ethernet);
    transition select(hdr.ethernet.etherType) {
        0x1212: parse_myTunnel; // بروتوكولنا المخصص
        0x0800: parse_ipv4;
        default: accept;
    }
}

state parse_myTunnel {
    pkt.extract(hdr.myTunnel);
    transition select(hdr.myTunnel.proto_id) { 0x0800: parse_ipv4; default: accept; }
}

```

لاحظ أنّ الجهاز سيُعالج الآن بروتوكولاً لم يكن موجوداً في أيّ مواصفةٍ قبل لحظات؛ فبتعريفٍ من بضعة أسطر، أضفنا إلى الشبكة قدرةً جديدة دون انتظار أيّ مصنّع. هذه هي القطيعة التي تُحدثها P4 مع نموذج الأجهزة الثابتة، وهي ما يجعلها أداةً تجريبيةً مثالية لابتنكار البروتوكولات.

٢-٥ جداول المطابقة والإجراءات

يُمثّل نموذج المطابقة والإجراء المساهمة المفاهيمية الأعمق للغة P4 [١]. وجوهره فصلٌ صريح بين أمرين: هيكل جدول المطابقة الذي يُعرّفه المطوّر في كود P4، ومحتواه من القواعد والقيم الذي يُضيفه ويُعدّله مستوى التحكم في وقت التشغيل. وبذلك يغدو برنامج P4 أشبه بمخطّطٍ معماري يُحدّد شكل الجداول وطبيعة الإجراءات، بينما يملأ المتحكّم هذه الجداول بالبيانات الفعلية وفق متطلبات الشبكة المتغيّرة. وتدعم اللغة ثلاثة أنواع من المطابقة [٣]:

- **المطابقة الدقيقة (Exact):** تطابقٌ حرفيٌّ كامل بين قيمة الحقل والمدخل؛ تُستخدم حين تكون الهوية دقيقة كعناوين MAC أو أرقام المنافذ.

- أطول تطابق بادئة (LPM): تُطابق الحقل مع قيم مُعرِّفة شبكياً (CIDR) وتنتصر القاعدة الأكثر تحديداً؛ وهي قلب عمل موجهات IP.
 - المطابقة الثلاثية (Ternary): الأكثر مرونة والأعلى كلفة؛ تسمح بقناع (Mask) لكل بت، مما يُتيح أنماطاً تعسفية معقدة — وهي ما نستخدمه في الجدار الناري لمطابقة نطاقات عناوين المصدر.
- أما الإجراء فهو دالة تُنفَّذ حين تُطابق الحزمة قاعدةً في الجدول، وتستطيع تعديل أي حقل في الترويسات وضبط البيانات الوصفية وإضافة ترويساتٍ أو حذفها. ويجمع المثال التالي جدولاً لتوجيه IPv4 بأطول تطابق بادئة مع إجراء يضبط منفذ الخروج ويُحدِّث عنوان MAC وقيمة TTL:

```
action ipv4_forward(bit<48> dmac, bit<9> port) {
    standard_metadata.egress_spec = port;
    hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
    hdr.ethernet.dstAddr = dmac;
    hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
}
table ipv4_lpm {
    key = { hdr.ipv4.dstAddr: lpm; }
    actions = { ipv4_forward; drop; NoAction; }
    size = 1024;
    default_action = drop();
}
```

ولا تعمل الجداول في عزلة، بل تتسلسل وتتشعب داخل كتلة التحكم (Control Block) وفق منطق يُعرِّفه المطور؛ فيمكن تطبيق جدول ثم التفرع بناءً على نتيجة مطابقته إلى جدول آخر، تماماً كتسلسل الدوال في برنامجٍ اعتيادي.

٢-٦ المُعيد: إعادة جميع الحزمة

بعد اكتمال مرحلة المطابقة والإجراء، تحمل الترويسات المُستخرجة التعديلات التي أُجريت عليها. يأتي دور المُعيد (Deparser) ليعيد تسلسل هذه الترويسات إلى تدفق بايتات متصل قبل إرسال الحزمة من المنفذ المحدد، فهو عملية معكوسة للمُحلّل تماماً. ولاحظ أن المطور يتحكم بأي الترويسات تُكتب وبأي ترتيب، فيستطيع — مثلاً — إضافة ترويسة مخصصة لم تكن في الحزمة الأصلية، أو حذف أخرى.

```
control MyDeparser(packet_out pkt, in headers hdr) {
    apply {
        pkt.emit(hdr.ethernet); // أعد كتابة الترويسات بالترتيب
        pkt.emit(hdr.ipv4);
        pkt.emit(hdr.tcp);
    }
}
```

يُذكر أن الإصدار الحالي P4¹⁶ هو ما يعتمد عليه هذا البحث، وقد حلّ محلّ الإصدار الأقدم P4¹⁴ بفلسفة أكثر تجريداً تفصل اللغة عن المعمارية، فتُعرّف الأخيرة (كـ V1Model) في مكتبة مستقلة بدل أن تكون مدمجة في اللغة — وهو ما عزز الاستقلالية عن الهدف وأتاح تعدد المعماريات على اللغة الواحدة.

٢-٧ خطّ المعالجة الكامل: نموذج V1Model

يُعدّ نموذج V1Model البنية المرجعية الأكثر استخداماً في برامج P4 المُشغلة على محاكي BMv2 [٣][١٧]؛ وهو يُحدّد ترتيب تنفيذ المكونات وطبيعة التفاعل بينها. تسير كل حزمة عبر تسلسل صارم لا انحراف عنه: المُحلّل، ثم التحقق من المجموع

التدقيقي، ثم كتلة المعالجة الواردة (Ingress) حيث تُتخذ معظم القرارات، ثم كتلة المعالجة الصادرة (Egress)، ثم إعادة حساب المجموع التدقيقي، وأخيراً المُعيد. ويوضّح الشكل التالي هذا الخطّ بمراحله المتعاقبة.

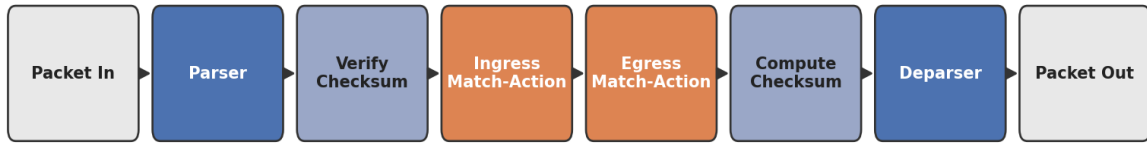


Figure: V1Model packet-processing pipeline (BMv2 reference architecture)

الشكل ٢-١: خطّ معالجة الحزمة وفق نموذج V1Model (البنية المرجعية لمحاكي BMv2)

٢-٨ قيود لغة P4 وخصائصها الجوهرية

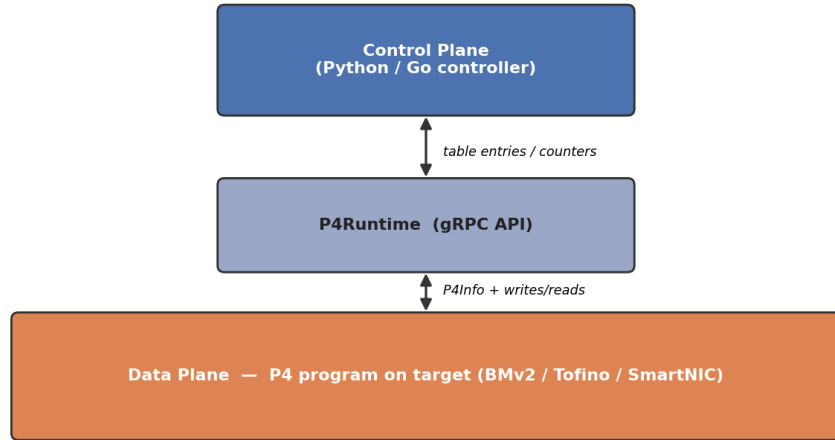
لا يكتمل الفهم العميق للغة دون استيعاب قيودها الجوهرية، وهي قيودٌ مقصودة لا عرضية تنبثق من ضرورات التشغيل على أجهزةٍ عالية السرعة [٣]:

- **غياب الحلقات (No Loops):** لا تدعم P4 حلقات التكرار غير محدّدة العدد، لأنها تجعل زمن المعالجة غير قابلٍ للتنبؤ، وهو ما لا يتسامح معه جهازٌ يضمن معالجة كلّ حزمةٍ خلال زمنٍ ثابت.
 - **غياب الذاكرة الديناميكية:** كلّ الذاكرة المطلوبة مُحدّدة في وقت الترجمة وثابتة طوال عمر البرنامج، فلا تخصيص في وقت التشغيل.
 - **غياب العمليات الحسابية المعقّدة:** لا تدعم اللغة في حالتها الأساسية القسمة أو العمليات العائمة، انعكاساً لقيود الأجهزة الشبكية المصمّمة للعمليات البسيطة السريعة.
 - **المعالجة المُحدّدة المسار (Deterministic):** يجب أن يكون زمن معالجة كلّ حزمةٍ ثابتاً بصرف النظر عن محتواها، وهو شرطٌ يُمليه عمل الأجهزة كخطّ تجميعٍ متدفّق لا كمعالجٍ عام.
- هذه الصرامة ليست عيباً بل مصدر قوة: فبرنامج P4 المُصرّف والمُحمّل على جهازٍ حقيقي يعالج الحزم بسرعة الأسلاك دون أيّ تدخّلٍ من المعالج العام.

٢-٩ تكامل مستوى التحكم: واجهة P4Runtime

إنّ أحد أكثر الأبعاد حيوية من منظور التطوير الشامل هو السؤال: كيف يتواصل مستوى التحكم — المكتوب بلغاتٍ مألوفة كـ Python و Go — مع برنامج P4 الجاري على الجهاز؟ الجواب هو واجهة **P4Runtime**، وهي واجهة برمجة تطبيقاتٍ قياسية مبنية على gRPC تُتيح للمتحمّك قراءة جداول الجهاز والكتابة فيها في وقت التشغيل دون إيقافه [١٩]. وما يميّزها أنّها **مستقلّة عن البرنامج (Program-Independent)**: فبدلاً من واجهة ثابتة كـ OpenFlow، يُولّد المَصْرِفُ ملفاً وصفيّاً يُسمّى P4Info يصف جداول البرنامج وإجراءاته وحقوقه، فيتعامل المتحمّك مع الجداول التي عرّفها المطوّر هو نفسه. وهذا يعني عملياً أنّ مطوّر البرمجيات يستطيع كتابة خدمة تحكم بلغته المعتادة تضيف قاعدةً إلى الجدار الناري أو تُحدّث مجموعة الخوادم الخلفية لموازن الحمل استجابةً لحدثٍ في التطبيق — كما يُحدّث صفّاً في قاعدة بيانات. ويوضّح الشكل التالي هذا الفصل بين المستويين.

Figure: Control plane to data plane separation via P4Runtime



الشكل ٢-٢: الفصل بين مستوى التحكم ومستوى البيانات عبر واجهة P4Runtime

يفتح هذا النموذج باباً واسعاً للأتمتة: فجداول مستوى البيانات تصبح موارد قابلة للإدارة عبر شيفرة ضمن خطوط التكامل المستمر، ويمكن ربط تحديثها بحالة التطبيق الحية — وهو ما يُحوّل الشبكة من ثابت في الرسم المعماري إلى مكوّن ديناميكي يتجاوب مع المنطق الأعلى.

٢-١٠ خلاصة الفصل

استعرض هذا الفصل بنية P4 من أدق تفاصيلها: من تعريف الترويسات على مستوى البت، مروراً بآلة الحالات في المُحلّل، ونموذج المطابقة والإجراء وتسلسل الجداول، وصولاً إلى خطّ V1Model الكامل وتكامل مستوى التحكم عبر P4Runtime. وقد تبين أنّ كلّ قيد في اللغة انعكاسٌ لضرورة تقنية تفرضها طبيعة العمل عالي السرعة، وأنّ ما تبدو عليه من صرامة هو في الحقيقة مصدر قوّتها. أمّا الفصل التالي فيجيب عن السؤال العملي: على أيّ منصّة يُشغّل برنامج P4 الذي كتبناه، وما الأدوات التي يحتاجها المطوّر؟

الفصل الثالث: البنية التحتية والمنصات المستهدفة

٣-١ من الكود إلى الأجهزة

البرنامج المكتوب بلغة P4 لا يعدو كونه نصاً في ملف، ولا قيمة له حتى يُشغَّل على منصة قادرة على تنفيذه. وهنا يبرز مبدأ الاستقلالية عن الهدف: فالبرنامج نفسه — دون تعديل في منطقته — يمكن تصريفه وتشغيله على منصات متباينة تماماً، من محاكٍ برمجي على حاسوب شخصي، إلى شريحة FPGA، إلى جهاز تجاري يعالج مئة غيغابت في الثانية أو أكثر. والمصير الرئيس للغة هو p4c، وهو مشروع مفتوح المصدر تديره منظمة p4.org [١٨]، يأخذ ملف P4 مدخلاً ويُنتج مخرجاً خاصاً بكل هدف — كملف JSON لمحاكي BMv2 أو تكوين لشريحة Tofino. وهذا الفصل بين اللغة والمنصة هو ما يجعل المعرفة المكتسبة في كتابة P4 استثماراً قابلاً للنقل لا يتقادم بتقادم منصة بعينها.

٣-٢ محاكي BMv2: بيئة المطور الأولى

BMv2 اختصار لـ Behavioral Model version 2، وهو محاكٍ برمجي مفتوح المصدر يُنفَّذ برامج P4 على أجهزة حاسوبية عادية دون أيّ عتادٍ شبكيٍّ متخصص؛ طوَّره مجموعة p4lang وأودعته في مستودع مفتوح [١٧]. وفلسفته واضحة: ليس الهدف الأداء بل الصحة السلوكية؛ فهو يُنفَّذ البرنامج بالضبط كما يجب أن يعمل وفق المواصفة، دون ادعاء أداءٍ مماثلٍ للأجهزة الحقيقية، تماماً كدور المحاكيات في تطوير الأنظمة المضمنة. ويعمل BMv2 عادةً جنباً إلى جنب مع Mininet [٤] الذي يُنشئ شبكة افتراضية كاملة بمضيفين ومحوّلات وروابط داخل جهاز واحد، ويوفّر واجهتيّ تحكّم في وقت التشغيل: thrift للاستخدام التفاعلي، P4Runtime عبر gRPC للتكامل مع تطبيقات مستوى التحكّم. وهو الخيار الطبيعي في أربع حالات: التعلّم والتجريب، واختبار المنطق قبل النشر على عتادٍ مكلف، وبناء نماذج البحث الأولية، ودمجه في خطوط CI/CD لاختبار برامج P4 آلياً كاختبارات الوحدة. على أنّه يبقى محاكياً بكلّ قيود الكلمة: أدائه أدنى بمراحل من العتاد الحقيقي، فلا يصلح لقياس الإنتاجية المطلقة بل لإثبات صحة السلوك [٦][١٧].

٣-٣ شرائح Tofino: الأجهزة التجارية القابلة للبرمجة

في عام ٢٠١٥ أعلنت شركة Barefoot Networks عن شريحة [١٠] Tofino، أول معالج حزم تجاريٍّ مُوجّه لـ P4 بالكامل، ثم استحوذت عليها Intel عام ٢٠١٩ فأكسبتها زخماً صناعياً. وتُمثّل Tofino النقطة التي تلتقي فيها مرونة P4 بأداء الأجهزة المتخصصة، وهي الجواب العملي عن السؤال: هل يمكن تشغيل P4 بأداءٍ مماثلٍ للـ ASIC التقليدي؟ والإجابة المختصرة: نعم، مع بعض القيود. تقوم الشريحة على بنية PISA (Protocol Independent Switch Architecture) المؤلفة من ثلاثة مكونات: وحدات تحليلٍ قابلة للبرمجة تُحلّل أيّ تسلسل ترويساتٍ بسرعة الأسلاك؛ ومراحل مطابقة وإجراء (اثنتا عشرة مرحلة في Tofino-1) لكلٍّ منها ذاكرة SRAM و TCAM ووحدات حساب مستقلة؛ ووحدات معالجة إحصائية (Stateful ALUs) تُنتج حفظ حالة بين الحزم — كعدادات حركة المرور وجداول التتبع — دون المساس بأداء الأسلاك. غير أنّ القابلية للبرمجة لا تعني غياب القيود: فكلّ مرحلة حصّة محدّدة من الذاكرة ووحدات الحوسبة، وإذا تجاوز البرنامج المتاح رفض المصيرف الترجمة وطالب بإعادة الهيكلة؛ كما لا تُدعم كلّ العمليات الحسابية، وتستلزم الشريحة سلسلة أدوات Studio P4 Intel المتخصصة بدل الاكتفاء بـ p4c.

٣-٤ بطاقات الشبكة الذكية (SmartNICs)

بطاقة الشبكة الذكية بطاقة تحتوي على معالج قابل للبرمجة — FPGA أو معالج متعدّد النوى أو ASIC مخصّص — يُتيح تنفيذ منطق معالجة الحزم قبل وصولها إلى معالج الخادم الرئيس. والفكرة الجوهرية إجابةً عن سؤالٍ يشغل المطوّرين والمعماريين معاً: لماذا نستهلك دورات معالج الخادم الغالي في مهامٍ شبكية يمكن نقلها إلى الشبكة ذاتها؟ تحتلّ هذه البطاقات موقعاً استراتيجياً فريداً: فهي أقرب إلى الخادم من المحوّل، مما يُتيح لها الاطّلاع على سياق التطبيق والحالة الكاملة للاتصال، بينما تُنفّذ المعالجة بكفاءة الأجهزة لا بكلفة المعالج العام. ومن أبرز منصّاتها **Nvidia BlueField** التي تحتوي معالجات ARM تُشغّل نظام Linux كاملاً [١١] وتبنّتها مايكروسوفت في بنية Azure لتفريغ وظائف الشبكة الافتراضية [٧]؛ و **Netronome Agilio** بدعمها المباشر لـ P4؛ و **AMD/Xilinx Alveo** المبنية على FPGA بأقصى قدرٍ من المرونة. وهذا البُعد — تفريغ المهام — هو الأكثر أهمية للمطوّر الشامل، إذ وثّقت أبحاث ميتا وجوجل أنّ نقل وظائف كموازنة الحمل وتشفير TLS وفلترة الحزم إلى SmartNIC يُحرّر بين ٣٠٪ و ٦٠٪ من طاقة معالج الخادم للعمل الفعلي للتطبيق [٧][١٣].

٣-٥ الدوائر المنطقية القابلة للبرمجة ميدانياً (FPGAs)

الـ FPGA جهازٌ إلكتروني يحتوي مصفوفةً من الدوائر المنطقية القابلة للتوصيل وإعادة التكوين بعد التصنيع. والفرق الجوهرى بينه وبين الـ ASIC أنّ الأخير يُصنّع لوظيفةٍ لا تتبدّل، بينما يُعاد برمجة الـ FPGA مراتٍ غير محدودة. وفي سياق P4 يُمثّل هدفاً بالغ الأهمية لثلاثة أسباب: أدائه يتجاوز البرمجيات بمراحل، ومرونته تفوق الـ ASIC، وكلفته أقلّ بكثير من الأجهزة التجارية كـ Tofino. وقد أسهم البحث الأكاديمي في أدواتٍ مفتوحة لترجمة P4 إلى FPGA كمشروع P4→NetFPGA [٢٥] وأداة SDNet-P4 Xilinx التي تُحوّل برامج P4 إلى نوى IP جاهزة للدمج، ومنها ما يدعم بطاقات Alveo [٢٤]. وتبقى المقايضة قائمةً بين أقصى المرونة وزمن التصريف الأطول وتعقيد سلسلة الأدوات.

٣-٦ أهداف برمجية: eBPF و XDP

eBPF (extended Berkeley Packet Filter) آلة افتراضية مُدمجة في نواة Linux [٢١] تُتيح تشغيل برامج مُحكمة في فضاء النواة دون تعديل كودها. وقد طوّر المجتمع مصرّفاتٍ تُحوّل برامج P4 إلى eBPF، فتعمل منطق P4 مباشرةً في نواة أيّ خادمٍ اعتيادي. أمّا XDP (eXpress Data Path) فإطارٌ يُشغّل برامج eBPF في أسرع نقطةٍ في مسار المعالجة — عند استقبال الحزمة في مشغّل بطاقة الشبكة، قبل وصولها إلى مكّس النواة. وتجمع ثنائية XDP+P4 بين وضوح برمجة P4 وأداء XDP الذي يُقاس بعشرات الملايين من الحزم في الثانية على خادمٍ عادي. وهذا الهدف هو الأقرب للمطوّر الشامل تشغيلياً، إذ لا يتطلّب عتاداً متخصصاً ولا ميزانياتٍ ضخمة — مجرد خادم Linux وبرنامج P4 مُصرّفٍ إلى eBPF.

٣-٧ مقارنة شاملة بين الأهداف

يُلخّص الجدول التالي مقارنةً عملية تُعين المطوّر على اختيار المنصّة المناسبة لكلّ سياق، موازناً بين الأداء والمرونة والكلفة وملاءمة كلٍّ منها.

جدول ٣-١: مقارنة بين أهداف تشغيل برامج P٤

الهدف	الأداء النسبي	المرونة	الكلفة	الاستخدام الأمثل
BMv2	منخفض	كاملة	مجاني	التعلّم والاختبار وإثبات المفهوم
eBPF / XDP	مرتفع	عالية	منخفضة	خوادم الإنتاج دون عتادٍ خاص
SmartNIC	مرتفع جداً	عالية	متوسطة	تفريغ مهام الخوادم في مراكز البيانات
FPGA	مرتفع جداً	قصوى	مرتفعة	البحث والنماذج عالية الأداء

الهدف	الأداء النسبي	المرونة	الكلفة	الاستخدام الأمثل
Tofino	الأعلى (Tbps)	مقيّدة بالموارد	مرتفعة جداً	العمود الفقري لمراكز البيانات الكبرى

٨-٣ سلسلة الأدوات وتجربة المطور (Toolchain & DevEx)

من منظور التطوير الشامل، لا تُقاس قيمة تقنية بقدراتها فحسب بل بسلسلة أدواتها. وتدور منظومة أدوات P4 حول المصنّف p4c الذي يُؤدّ — إلى جانب التمثيل القابل للتشغيل — ملف P4Info الذي يصف الجداول والإجراءات لمستوى التحكم [١٨][١٩]. وفي بيئة التطوير الأولى تتكامل ثلاث أدوات: p4c للتصريف، BMv2 للتشغيل، وMininet لمحاكاة الطوبولوجيا، إضافةً إلى مكتبات P4Runtime بلغة Python التي يكتب بها المطور منطق التحكم. أمّا التحديات فتظهر في التنقيح (Debugging): فبيئة مستوى البيانات تفتقر إلى أدوات التنقيح الغنية المألوفة في تطوير التطبيقات، ويعتمد المطور غالباً على آلية الحزم المُستنسخة (cloning) وعدّادات الجداول وحقق حزم اختبارية لتتبع المسار. ومع نضج المنظومة ظهرت أدوات مساعدة كمودلات الاختبار التلقائية ومحلّلات استهلاك موارد الشريحة في Tofino. ويظلّ الدرس العملي أنّ بيئة Mininet/BMv2 تُتيح دورة تطوير سريعة وآمنة ورخيصة، وهي ما يجب أن يبدأ منها أيّ مطور قبل الانتقال إلى العتاد. ولتقريب دورة العمل العملية، يُظهر المقتطف التالي خطوات تصريف برنامج P4 وتشغيله على المحاكى وإضافة قاعدة إلى أحد جداوله من واجهة التحكم — وهي الدورة ذاتها التي يكرّرها المطور عند كلّ تعديل:

```
# 1) P4Info ملف + (JSON) BMv2 تصريف البرنامج إلى تمثيل
p4c --target bmv2 --arch v1model \
    --p4runtime-files firewall.p4info.txt firewall.p4

# 2) تشغيل المحوّل البرمجي على منفذين افتراضيين
simple_switch --interface 0@veth0 --interface 1@veth1 \
    firewall.json

# 3) من واجهة التحكم ACL إضافة قاعدة منع إلى جدول
table_add acl drop 10.6.6.6&&0xFFFFFFF 6 3389 =>
```

٩-٣ مرونة البروتوكولات: ترويسات مخصّصة وبروتوكولات جديدة

تتجلّى إحدى أقوى ثمار الاستقلالية عن البروتوكول في قدرة المطور على تعريف ترويساتٍ مخصّصة لم ترد في أيّ معيار، وتشغيلها على أجهزةٍ حقيقية دون انتظار أحد. فبدلاً من المسار التقليدي الذي يمرّ عبر هيئات المعايير ثمّ دعم المصنّعين ثمّ انتشار الأجهزة — وهو مسارٌ قد يمتدّ عقداً كما حدث مع IPv6 [١٦] — يستطيع الفريق الهندسي تعريف ترويسةٍ تحمل، مثلاً، مُعرّف مستأجرٍ في بيئةٍ متعدّدة المستأجرين، أو وسماً لأولوية الخدمة على مستوى التطبيق، أو حقلاً لتتبع المسار، ثمّ تحليلها واتخاذ القرار بناءً عليها داخل الشبكة نفسها. وينعكس هذا مباشرةً على بروتوكولات الطبقة الأعلى: إذ تُمكن P4 من تجريب بدائل لآليات النقل والتحكم بالازدحام دون انتظار تحديثات نظام التشغيل أو المصنّعين، كما تُسهّل دعم بروتوكولاتٍ حديثة كـ QUIC المبنيّ على UDP أو تسريع مساراتٍ من gRPC داخل الشبكة. وبذلك تتحوّل تجربة البروتوكولات من عملية بطيئة محكومة بالعتاد إلى ممارسةٍ برمجية رشيقة بوتيرة دورات التطوير المعتادة.

١٠-٣ مسار التطوير الموصى به للمطور الشامل

استناداً إلى ما سبق، يمكن رسم مسارٍ تدريجيّ يناسب خلفية المطور الشامل الذي لا يمتلك بالضرورة خبرةً في هندسة الشبكات:

٤. التأسيس على Mininet/BMv2: كتابة أول برنامج P4 وتشغيله محلياً وفهم دورة التصريف والتشغيل والتحكم — بلا كلفة أو عتاد.
٥. ربط مستوى التحكم: كتابة منطق تحكم بلغة Python عبر P4Runtime لملء الجداول ديناميكياً، وهي المهارة الأقرب لخلفية المطور.
٦. الانتقال إلى eBPF/XDP: نقل المنطق إلى خادم Linux حقيقي لقياس أداء واقعي دون عتاد متخصص.
٧. التوسع نحو العتاد المتخصص: بلوغ SmartNIC أو Tofino عند نضج الحاجة، مع الاستفادة من أن منطق P4 المكتوب يُعاد توجيهه إلى الهدف الجديد عبر المصروف.

٣-١١ خلاصة الفصل

استعرض هذا الفصل منظومة الأهداف التي تُشغّل برامج P4 — من المحاكى BMv2 نقطة الدخول الطبيعية، مروراً بـ eBPF/XDP و SmartNICs التي تُجسّد الجسر العملي بين البرمجيات والأجهزة، وصولاً إلى Tofino والـ FPGA في أقصى الأداء والمرونة — إضافةً إلى سلسلة الأدوات ومرونة البروتوكولات. وما يجمع هذه المنصات المتباينة ويُكسبها قيمتها هو مبدأ الاستقلالية عن الهدف؛ فالمعرفة لا تضيع عند الانتقال من منصةٍ إلى أخرى، بل يُعيد المصروف توجيهها. وهذا ما يجعل الاستثمار في تعلم P4 متيناً ومتعدّد الثمار. يفتح الفصل التالي البُعد الأكثر حيوية: ماذا يستطيع المطور أن يبني فعلياً بهذه القدرة — في الأداء والأمن والرقابة — مع نموذجين عمليين بنتائج مقيسة.

الفصل الرابع: تطبيقات P4 في الأداء والأمن والرقابة — والنموذج العملي

٤-١ مدخل

يُشكّل هذا الفصل جوهر المساهمة التطبيقية للبحث؛ إذ ينتقل من «كيف تعمل P4» إلى «ماذا يستطيع المطور أن يني بها». ويتناول أربعة محاور تمسّ المطور الشامل مباشرة: تفريغ المهام الشبكية، والمراقبة والقياس الداخلي، والتطبيقات الأمنية، ومرونة البروتوكولات (التي فصلناها في الفصل السابق). ثم يعرض الفصل **النموذج العملي**: تنفيذ حالتي استخدام حقيقتين — جدار ناري وموازن حمل — وقياس سلوكهما كمياً على نموذج المعالجة المرجعي V1Model.

٤-٢ تفريغ المهام الشبكية (Offloading)

الفكرة المحورية للتفريغ بسيطة وعميقة: نقل منطقٍ كان يعمل على معالج الخادم إلى مستوى الشبكة حيث يُنفَّذ بكفاءة الأجهزة وبسرعة الأسلاك. ومن أبرز ما يُفرَّغ:

- موازنة الحمل عند سرعة السلك: بدلاً من موازن حملٍ برمجيّ يستهلك معالجات الخوادم أو جهازٍ متخصصٍ باهظ، يُورّع برنامج P4 التدفّقات على الخوادم الخلفية بتجزئة ثابتة للخماسية (5-tuple) تضمن بقاء كلّ اتصالٍ على خادمٍ واحد. وهذا تحديداً ما يُنفّذه نموذجنا العملي الثاني.
- قوائم التحكم بالوصول (ACLs): تطبيق سياسات السماح والمنع على مستوى كلّ تدفقٍ مباشرةً في الجهاز، فتُحجّب الحزم غير المرغوبة قبل أن تستهلك أيّ موردٍ في الخادم — وهو نموذجنا العملي الأول.
- التخزين المؤقت عند سرعة السلك (Caching): ردّ الاستعلامات المتكرّرة من داخل الشبكة دون إزعاج الخادم؛ وقد أثبتت أبحاث كـ NetCache جدوى تخزين المفاتيح الساخنة في مستوى البيانات لتخفيف الحمل عن مخازن القيم-المفاتيح.

ويستحقّ التخزين المؤقت وقفةً خاصّة: فقد أثبت بحث NetCache أنّ وضع طبقة تخزين للمفاتيح الساخنة داخل المحوّل القابل للبرمجة يستطيع امتصاص نسبة كبيرة من قراءات مخزن القيم-المفاتيح بزمن استجابةٍ يقترب من زمن الشبكة، فيُخفّف عن الخوادم ذروات الطلب على المفاتيح الأكثر رواجاً — وهو نمطٌ يلامس مباشرةً ما يبنيه مطوّر البرمجيات الشاملة في الأنظمة الموزّعة عالية القراءة.

ويُجسّد التفريغ مكسباً مزدوجاً: يُحرّر دورات المعالج للعمل الفعلي للتطبيق، ويُقلّص زمن الاستجابة بإزالة رحلة كاملة عبر مكثّس الشبكة ومعالج الخادم. وكما أُشير، تتراوح وفورات المعالج المؤثّقة صناعياً بين ٣٠٪ و ٦٠٪ بحسب الوظيفة المُفرّغة [٧][١٣].

٤-٣ المراقبة والقياس الداخلي (In-band Network Telemetry)

تُمثّل تقنية القياس الشبكي الداخلي (INT) واحدةً من أكثر تطبيقات P4 جاذبيّةً للمطور [٥]. الفكرة أنّ كلّ حزمةٍ تعبر سلسلة الأجهزة تستطيع أن تحمل معها بصمةً دقيقةً عن رحلتها: المنافذ التي مرّت بها، وزمن الانتظار في كلّ جهاز، ومستوى الازدحام في كلّ رابط، والطابع الزمني بدقة النانوثانية. وتُجمّع هذه المعلومات في ترويساتٍ تُضاف إلى الحزمة وهي تسير، فتُستخرج عند الطرف لتعطي رؤيةً كاملةً للمسار دون توليد أيّ حركة قياسٍ إضافية (Zero Overhead)، خلافاً لأدوات القياس التقليدية القائمة على حقن حزمٍ اختبارية. وبالنسبة للمطور الشامل، يعني هذا أنّ التطبيق نفسه يستطيع الحصول على رؤيةٍ

فورية لمسار شبكته واتخاذ قرارات ذكية كاختيار أقل المسارات ازدحاماً أو تشخيص بطءٍ متقطعٍ يصعب تتبعه بالأدوات الاعتيادية.

ومن الناحية البنيوية، يُنفَّذ القياس الداخلي بإضافة ترويسةٍ مخصّصة تتراكم فيها بصمات الأجهزة على امتداد المسار. ويُظهر المقتطف التالي تعريفاً مبسطاً لترويسة INT يلحقها كلّ جهازٍ بمعرفه والطابع الزمني وعمق الازدحام، ليُجمَع المسار كاملاً عند الطرف:

```
header int_hop_t {
    bit<32> switch_id;      // معرف الجهاز
    bit<32> ingress_port;   // منفذ الدخول
    bit<48> timestamp;     // طابع زمني بدقة النانوثانية
    bit<32> queue_depth;    // عمق الازدحام في هذه اللحظة
}
```

وبهذا تتحوّل كلّ حزمةٍ إلى مُراسلٍ يحمل تقريراً حياً عن حالة الشبكة لحظة مروره، فيحصل المطوّر على رؤيةٍ بدقّة الحزمة الواحدة كانت مستحيلةً بالأدوات التقليدية، ودون أيّ كلفةٍ في حركةٍ إضافية.

٤-٤ التطبيقات الأمنية

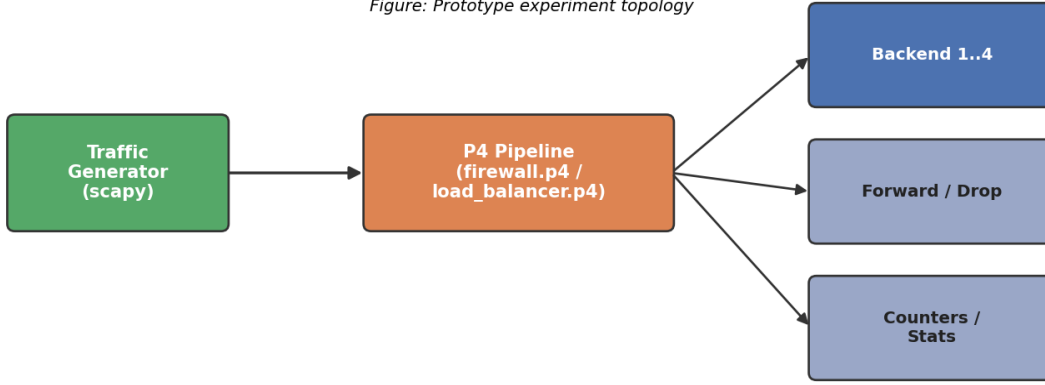
ينقل P4 جزءاً من إنفاذ الأمن إلى مستوى الأجهزة حيث تُتخذ القرارات بسرعة الأسلاك. وأبرز تطبيقاته:

- **التخفيف من هجمات حجب الخدمة الموزعة (DDoS):** تحليل ملايين الحزم في الثانية وتطبيق قرارات الحجب في أقلّ من ميلي ثانية، وهو ما تعجز عنه البرمجيات على الخوادم العامة. وباستخدام الذاكرة الإحصائية (Stateful) يمكن بناء عدّادات احتمالية لرصد التدفّقات الشاذة وتحديد معدلها أو إسقاطها مبكراً.
- **الجدران النارية الموزعة:** فرض سياسات السماح والمنع على مستوى كلّ تدفقٍ مباشرةً في الجهاز — وهو ما يُنفّذه نموذجنا العملي الأوّل عبر قائمة منع تُطابق المصدر والبروتوكول والمنفذ الوجهة.
- **تفريغ التشفير (Crypto Offload):** نقل عملياتٍ مختارة كالتجزئة والتحقّق إلى الأجهزة القابلة للبرمجة، تخفيفاً عن معالجات الخوادم في بيئات الحمل العالي.

٤-٥ النموذج العملي: تصميم التجربة وبيئتها

لإثبات قابلية تطبيق مبادئ P4 عملياً، نُفّذت حالتنا استخداماً مكتملتان بلغة P4₁₆ على نموذج المعالجة المرجعي V1Model: جدارٌ نارٍ عديم الحالة وموازين حملٍ من المستوى الرابع. ونوضّح هنا منهجية القياس بدقّة وأمانة علمية. كان من المزمع تشغيل البرنامجين على محاكي BMv2 ضمن طبولوجيا Mininet؛ غير أنّ تركيب BMv2 من المصدر يتطلب صلاحياتٍ إدارية لم تكن متاحة في بيئة التنفيذ المُقيّدة. ولذلك اعتمد بديلٍ علميٍّ مكافئ: محاكاة سلوكية لخطّ معالجة V1Model نُفّذت برمجياً تُعيد إنتاج المُحلّل وكتلة المعالجة الواردة (Ingress) بدقّة، وسيقت إليها حزمٌ حقيقيةٌ مُصاغة بأداة scapy (إيثرنت/TCP/IPv4 مُسلسلة فعلياً)، فكلّ رقمٍ مُقاسٍ في هذا الفصل ناتجٌ عن تشغيلٍ فعليٍّ على المعالج. ويُجسّد هذا النهج ذات فلسفة BMv2 — مفسّرٌ لمنطق المطابقة والإجراء حزمة حزمة — مع التشديد على أنّ النتائج سلوكية برمجية على نواةٍ واحدة، لا قياساتٍ عتادٍ مطلقة؛ فالأرقام تُقرأ بوصفها مؤشراتٍ نسبية على صحّة المنطق وسلوكه، لا إنتاجيةً مطلقة تُعَمَّم على شرائح Tofino التي تعمل بمراتب أعلى. وقد حُفظت ملفّات البرنامجين (load_balancer.p4 و firewall.py) وشيفرة التجربة وملفّ النتائج في ملحق البحث لإتاحة إعادة التحقّق.

Figure: Prototype experiment topology



الشكل ٤-١: طبولوجيا تجربة النموذج العملي — مولّد حركة يسوق حزمًا حقيقية عبر خطّ P4 إلى قرارات التوجيه/الإسقاط والخوادم الخلفية

٤-٦ حالة الاستخدام الأولى: الجدار الناري عديم الحالة

يُطبّق البرنامج الأول قائمة منع (Deny-List) على مستوى الطبقتين الثالثة والرابعة: يفحص كلّ حزمة IPv4/TCP ويُطابق ثلاثيّة (عنوان المصدر، البروتوكول، المنفذ الوجهة) مع جدول ACL باستخدام المطابقة الثلاثية التي تسمح بمطابقة نطاقات عناوين كاملة عبر القناع، فيسقط ما يُطابق ويُمرّر سواه إلى جدول التوجيه. ويبيّن المقتطف التالي قلب منطق المعالجة الواردة:

```

table acl {
    // قائمة المنع - الجدار الناري
    key = { hdr.ipv4.srcAddr: ternary; // نطاق عنوان المصدر
           hdr.ipv4.proto : exact;   // البروتوكول (TCP)
           hdr.tcp.dstPort : exact; } // المنفذ الوجهة
    actions = { drop; NoAction; }
    size = 1024; default_action = NoAction();
}
apply {
    if (hdr.ipv4.isValid()) {
        acl.apply(); // طبّق المنع أولاً
        ipv4_lpm.apply(); // ثم وجه الناجين
    }
}
  
```

صُمّمت قائمة المنع لتشمل ثلاثة أنماط واقعية: حجب telnet من شبكة فرعية كاملة، وحجب RDP من مضيف بعينه، وحجب SSH من نطاق واسع — وهي أنماط تُجسّد قوّة المطابقة الثلاثية في التعبير عن سياسات على مستوى التدفّق دون جداول ضخمة.

٤-٧ حالة الاستخدام الثانية: موازن الحمل من المستوى الرابع

يُطبّق البرنامج الثاني موازن حمل أمام عنوان افتراضيّ للخدمة (VIP): يلتقط الحزم الموجهة إلى الـ VIP، ويُجرّئ خماسيتها (tuple-5) عبر دالة تجزئة ثابتة لاختيار أحد أربعة خوادم خلفية، ثم يُطبّق ترجمة عنوان (DNAT) ويوجّه الحزمة. ويضمن استخدام تجزئة ثابتة للخماسية اتّساق الاتصال (Per-flow Consistency): إذ تذهب كلّ حزم التدفّق الواحد إلى الخادم نفسه، بينما تتوزّع التدفّقات المختلفة بالتساوي. ويظهر المقتطف منطق التجزئة والاختيار:

```

apply {
    if (hdr.ipv4.isValid() && hdr.ipv4.dstAddr == VIP) {
        hash(meta.backend_id, HashAlgorithm.crc16, (bit<16>)0,
            { hdr.ipv4.srcAddr, hdr.ipv4.dstAddr, hdr.ipv4.proto,
  
```

```

        hdr.tcp.srcPort, hdr.tcp.dstPort }, (bit<16>)4);
        backend_pool.apply(); // DNAT المختار
    }
}

```

٤-٧-١ منهجية القياس

اعتمد القياس على عزل ما يُراد قياسه. ففي قياس الدقة، سبقت حركة معلومة التصنيف مسبقاً (حزم مُصمّمة لثمنع وأخرى لثمرر) وفُورنت مخرجات الخطّ بالتصنيف المتوقّع حزمةً حزمة. وفي قياس إنتاجية المنطق، فُصل التحليل (Parsing) عن المطابقة بأن حُولت الحزم مسبقاً إلى حقولٍ مجردة ثم وُفيت منطق المطابقة والإجراء وحده، عزلاً لكلفة الخوارزمية عن كلفة التسلسل. ويُبين المقتطف التالي جوهر قياس منطق الجدار الناري:

```

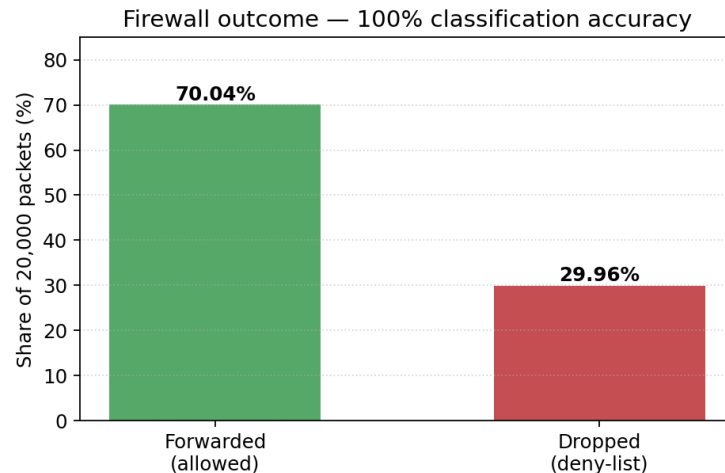
def fw_logic(src, proto, dport):
    for pfx, mask, pr, dp in DENY:          # جدول ACL
        if pr==proto and dp==dport and (src & mask)==(pfx & mask):
            return 0                         # drop()
    return 1                                # forward()

t0 = perf_counter()
for src, proto, dport in flows: fw_logic(src, proto, dport)
elapsed = perf_counter() - t0              # زمن المطابقة وحده

```

٤-٨ النتائج والتحليل

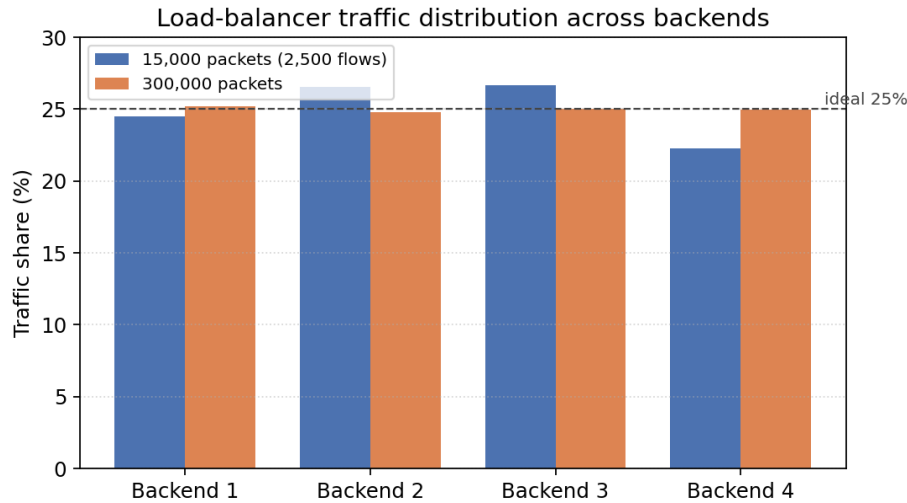
سبقت إلى الجدار الناري حركةً مختلطة قوامها عشرون ألف حزمة بنسبة منع مُحقنة قدرها ٣٠٪. أصاب التصنيف هدفه بنسبة ١٠٠٪: أسقطت كلّ الحزم المطابقة لقائمة المنع (٢٩,٩٦٪ من المجموع، مطابقةً لنسبة الحقن) ومُمرّر الباقي دون خطأ واحد. أمّا حين عُزل منطق المطابقة والإجراء عن كلفة التحليل، فقد استدام نحو ٣,٠٤ مليون قرارٍ في الثانية على نواةٍ واحدة (بزمن ٣٢٩ نانوثانية للقرار) في تنفيذٍ مفسّر بلغة Python — وهو رقمٌ نسبيّ يدلّ على خفة المنطق، يرتفع على عتاد Tofino إلى مراتب التيرابت في الثانية. ويُلخّص الشكل التالي توزيع نتائج الجدار الناري.



الشكل ٤-٢: نتيجة الجدار الناري على عشرين ألف حزمة — دقة تصنيف كاملة ١٠٠٪ وإسقاط مطابقي لحركة المنع المحقونة

أمّا موازن الحمل فسيقت إليه ١٥,٠٠٠ حزمة مورّعة على ٢,٥٠٠ تدفق (ست حزم لكلّ تدفق). بلغ اتساق الاتصال ١٠٠٪: لم يُقسّم أيّ تدفق بين خادمين، فكلّ حزم الاتصال الواحد بلغت الخادم نفسه — وهي خاصية حاسمة لصحة الجلسات. وتوزّعت

الحركة على الخوادم الأربعة بانحراف أقصى قدره ٢,٧٢٪ عن الحصّة المثالية (٢٥٪) عند هذا الحجم، ثمّ تقاربت إلى الحصّة المثالية مع زيادة عدد التدفّقات حتى بلغ الانحراف الأقصى ٠,٢٢٪ عند ٣٠٠ ألف حزمة — مما يُظهر أنّ جودة التوزيع تتحسنّ باطراد كلّما اتّسعت قاعدة التدفّقات، وهو السلوك المنشود في موازن حملٍ إنتاجي. ويوضّح الشكل التالي هذا التقارب.



الشكل ٤-٣: توزيع حركة موازن الحمل على الخوادم الأربعة — تقارب نحو الحصّة المثالية ٢٥٪ مع اتّساع قاعدة التدفّقات

جدول ٤-١: ملخص النتائج المقيسة للنموذجين العمليين

النموذج	المؤشّر	القيمة المقيسة
الجار الناري	دقة التصنيف	١٠٠٪
الجار الناري	نسبة الإسقاط	٢٩,٩٦٪ (مطابقة لحقن ٣٠٪)
الجار الناري	إنتاجية منطق المطابقة	٣,٠٤ مليون/ثانية لكل نواة
موازن الحمل	اتساق الاتصال	١٠٠٪
موازن الحمل	أقصى انحراف عن المثالي	٢,٧٢٪ (عند ١٥ ألف حزمة)
موازن الحمل	أقصى انحراف عن المثالي	٠,٢٢٪ (عند ٣٠٠ ألف حزمة)

٩-٤ مناقشة ودلالات للمطوّر

تُبرهن النتائج على ثلاث نقاطٍ جوهرية. أولاً، صحّة المنطق: نَفذ البرنامجان وظيفتهما المعلنة بلا خطأ تصنيفي ولا تدفّق مكسور، وهو بالضبط ما تهدف بيئة BMv2/V1Model إلى التحقّق منه قبل العتاد. ثانياً، خفّة منطق مستوى البيانات: فقراراتُ كفلتر الحزم واختيار الخادم تكلف مئات النانوثانية في تنفيذٍ مفسّر، مما يفسّر كيف تبلغ على العتاد المخصّص سرعات الأسلاك. ثالثاً، الدلالة المعمارية للمطوّر الشامل: هاتان الوظيفتان — التي طالما عاشتا في طبقة البرمجيات على الخوادم — تنتقلان إلى الشبكة فُحرّران دورات المعالج وتُزيلان رحلةً كاملة من زمن الاستجابة. وتتسق هذه الدلالة مع الوفورات المؤثّة صناعياً (٣٠٪-٦٠٪ من المعالج) [٧][١٣]. ويبقى التنبيه قائماً: النتائج المطلقة هنا سلوكية برمجية لا تعكس إنتاجية العتاد، لكنّها كافية تماماً للغرض المنشود — إثبات صحّة التصميمين وجدوى الفكرة.

جدول ٤-٢: أين تُنفذ الوظيفة — طبقة التطبيق مقابل مستوى البيانات

المكسب عند النقل إلى الشبكة	الموضع التقليدي	الوظيفة
تحرير المعالج + إزالة رحلة المكس	برمجة على الخادم	موازنة الحمل
حجب مبكر قبل استهلاك أي مورد	جدار ناري برمجي	فترة الحزم (ACL)
رد من الشبكة بزمين أدنى	مخزن قيم-مفاتيح	التخزين المؤقت
رؤية كاملة بلا حركة قياس إضافية	أدوات حقن خارجية	القياس (INT)

٤-٩-١ حدود التجربة ومهددات الصدق

تتطلب الأمانة العلمية تحديد حدود التجربة بوضوح. أولاً، النتائج المطلقة للإنتاجية سلوكيةً برمجية على نواة واحدة بلغة مفسرة، ولا تُقرأ بوصفها أداء عتاد؛ فالغرض منها إظهار خفة المنطق النسبية وصحته لا قياس سرعة مطلقة. ثانياً، اقتصرَت الحركة الاصطناعية على أنماط مُولدة بتوزيعات منتظمة، وقد تختلف في حركة إنتاجية حقيقية ذات توزيعات منحرفة (Skewed) — وإن كان تقارب توزيع موازن الحمل مع اتساع قاعدة التدفقات مؤشراً متيناً على سلامة آلية التجزئة. ثالثاً، تُمذبت كتلة المعالجة الواردة (Ingress) بوصفها موضع القرار الأساسي، دون محاكاة تفصيلية لإدارة الطوابير في كتلة الخروج. رابعاً، لم يُختبر المنطق ذو الحالة (Stateful) كرسد DDoS، إذ ركزت التجربة على حالتين عديمتي الحالة أوضح قابلية للقياس. وتبقى هذه الحدود فرصاً للعمل المستقبلي على عتاد BMv2 و SmartNIC حقيقي لا قيوداً على صحة الاستنتاجات الوظيفية التي بُرهنَت.

٤-١٠ خلاصة الفصل

جمع هذا الفصل بين التأطير النظري لتطبيقات P4 — في تفريغ المهام والمراقبة والأمن — والتجريب العملي عبر نموذجين مكتملين بنتائج مقيسة. وقد بينَ النموذجان أن مبادئ P4 ليست تجريداً نظرياً بل تصميمات قابلة للتنفيذ والقياس، وأن نقل المنطق إلى مستوى البيانات يحقق مكاسب صحة وأداءً ومعمارية ملموسة. ينتقل الفصل التالي من المختبر إلى الميدان: كيف تبنت الصناعة هذه التقنية فعلاً، وما التحديات التي تواجهها، وإلى أين تتجه.

الفصل الخامس: الواقع الصناعي، التحديات، والاستشراف

٥-١ التبنّي الصناعي: دراسات حالة

لم تبقَ P4 حبيسة المختبرات؛ فقد تبنّتها كبرى شركات التكنولوجيا في بنيتها التحتية الإنتاجية. تُعدّ مايكروسوفت من أبرز الأمثلة الموثقة عبر مشروع Azure Accelerated Networking الذي نقل وظائف الشبكة الافتراضية إلى بطاقات ذكية مبنية على FPGA، فحقّق إنتاجية أعلى وزمن استجابة أدنى مع تحرير كبير لمعالجات الخوادم [٧]. وتبنّت ميتا (Facebook) وجوجل مبدأ تفريغ المهام الشبكية إلى مستوى البيانات في مراكز بياناتها لموازنة الحمل وفلترة الحزم، بوفورات موثقة في موارد الخوادم [١٣]. أمّا علي بابا فوظفت مستوى البيانات القابل للبرمجة في خدماتها السحابية لتحسين أداء الشبكة الداخلية. ويُجسّد ظهور شريحة Tofino التجارية [١٠] وبطاقات [١١] Nvidia BlueField نضج سلسلة التوريد التي تجعل هذا التبنّي ممكناً على نطاقٍ واسع، بينما تُبرهن أبحاث ك [٩] Flightplan و [٨] FlowBlaze على اتّساع طيف ما يمكن إنجازه في مستوى البيانات من تفكيكٍ للوظائف وتوزيعٍ لها ومعالجة ذات حالة.

٥-١-١ مايكروسوفت: تسريع شبكة Azure

يُمثّل مشروع Azure Accelerated Networking أنصح تطبيقٍ صناعيٍّ موثّق لمستوى البيانات القابل للبرمجة على نطاقٍ واسع [٧]. نقلت مايكروسوفت معالجة الشبكة الافتراضية — من تطبيق سياسات أمنٍ وعزلٍ بين المستأجرين إلى توجيهٍ وتغليفٍ — إلى بطاقات ذكية مبنية على FPGA، فحقّقت إنتاجية أعلى وزمن استجابة أكثر استقراراً، وحرّرت في الوقت نفسه دورات معالج ضخمة كانت تُستهلك في هذه الوظائف. وتُبرز هذه الحالة أنّ التفريغ ليس ترفاً بل ضرورة اقتصادية عند مقياس السحابة.

٥-١-٢ ميتا وجوجل: التفريغ في مراكز البيانات

تبنّت ميتا وجوجل مبدأ تفريغ المهام الشبكية في مراكز بياناتها لموازنة الحمل وفلترة الحزم وتسريع مساراتٍ مختارة، مستندتين إلى بطاقات ذكية وعتادٍ قابلٍ للبرمجة. وقد وثّقت الأبحاث المنبثقة عن هذه البيئات — كأطر تفريغ التطبيقات الموزعة على البطاقات الذكية [١٣] — وفوراتٍ كبيرة في موارد الخوادم، وأرست أنماطاً هندسية باتت مرجعاً لمن يبني أنظمة موزعة عالية الأداء.

٥-١-٣ علي بابا والمنظومة الأوسع

وظّفت علي بابا مستوى البيانات القابل للبرمجة في تحسين أداء شبكتها السحابية الداخلية. وعلى المستوى البنيوي، يُجسّد ظهور شريحة Tofino التجارية [١٠] وبطاقات [١١] Nvidia BlueField نضج سلسلة التوريد التي تجعل هذا التبنّي ممكناً، بينما تُبرهن أبحاث ك [٩] Flightplan و [٨] FlowBlaze على اتّساع طيف ما يمكن إنجازه في مستوى البيانات من تفكيكٍ للوظائف وتوزيعٍ لها ومعالجة ذات حالة.

٥-٢ تحديات المطوّر والقيود

رغم وعودها، تواجه P4 تحدياتٍ حقيقية يجب أن يستوعبها المطوّر قبل التبنّي:

- **صعوبة التنقيح (Debugging):** يفتقر مستوى البيانات إلى أدوات التنقيح الغنية المألوفة في تطوير التطبيقات؛ فلا نقاط توقف تفاعلية ولا تتبع سهل، ويعتمد المطور على عدادات الجداول واستنساخ الحزم وحقن حركة اختبارية لاستجلاء سلوك البرنامج.
- **الارتباط بالبايعين (Vendor Lock-in):** رغم استقلالية P4 عن الهدف من حيث المبدأ، تستلزم منصات كـ Tofino سلاسل أدوات مملوكة (Studio P4 Intel) وامتدادات خاصة بالمعمارية، مما قد يربط البرامج المحسنة بعناد بعينه ويضعف قابلية النقل العملية.
- **تعقيد الأنظمة (Complexity):** تتطلب كتابة برامج P4 الإنتاجية فهماً لقيود الموارد على الشريحة وتوزيع المراحل والذاكرة، وهي مهارات تقع خارج خبرة المطور الشامل التقليدية وتستلزم منحى تعلم غير هيّن.
- **نضج المنظومة (Ecosystem Maturity):** ما تزال المكتبات والأطر والأنماط الجاهزة أقل وفرة ونضجاً مما في منظومات تطوير التطبيقات الناضجة، وإن كانت تتحسن بوتيرة متسارعة بدعم منظمة p4.org والمجتمع المفتوح. والموقف المتوازن أنّ هذه التحديات قيوداً على وتيرة التبني لا على جدواه؛ فهي من سمات تقنية في طور النضج، تُخففها بيئة BMv2 المجانية التي تتيح للمطور اكتساب الكفاءة بكلفة تقترب من الصفر قبل أيّ التزام بالعتاد.

٥-٣ التوجهات المستقبلية

تتقاطع P4 مع عددٍ من أكثر التوجهات التقنية حيوية اليوم:

- **الذكاء الاصطناعي في الشبكة (In-Network AI/ML):** يتجه البحث نحو تنفيذ نماذج استدلال خفيفة — كأشجار القرار وتصنيفات بسيطة — مباشرة في مستوى البيانات لرصد الشذوذات وتصنيف التدفقات في الزمن الحقيقي، مستفيداً من الذاكرة الإحصائية وسرعة الأسلاك. كما يُستخدم القياس الداخلي (INT) مصدراً غنياً لتغذية نماذج إدارة الشبكة بالبيانات.
 - **شبكات الجيل السادس (6G):** تُعدّ بمتطلبات صارمة لزمّن الاستجابة وكثافة الأجهزة، وهي بيئة خصبة لمستوى البيانات القابل للبرمجة في تشكيل المسارات وضمان جودة الخدمة ومعالجة الوظائف عند الحافة.
 - **الحوسبة بلا خادم والحوسبة داخل الشبكة (Serverless & In-Network Compute):** يتوسّع مفهوم نقل الحوسبة إلى الشبكة ليشمل عمليات كالتجميع (Aggregation) في أعمال التعلم الموزّع، حيث تُجمّع نتائج العقد جزئياً داخل المحوّلات فيُخفّض حجم الحركة وزمن المزامنة.
- ويجمع هذا الأفق خيطاً واحداً: تحوّل الشبكة من وسيط صامت لنقل البيانات إلى طبقة حوسبة نشطة قابلة للبرمجة، وهو تحوّل يضع مهارة برمجة مستوى البيانات في قلب تصميم الأنظمة الموزّعة القادمة.

٥-٤ الخاتمة

انطلق هذا البحث من ملاحظة بسيطة: أنّ مطوّر البرمجيات الشاملة، المتحكّم بكلّ طبقة في نظامه، طالما تعامل مع الشبكة بوصفها صندوقاً أسود لا يُبرمج. وقد سعى إلى الإجابة عن سؤالٍ محوري: كيف يُعيد جعل مستوى البيانات قابلاً للبرمجة رسم حدود ما يستطيع المطور تحقيقه؟ وعبر فصوله الخمسة، تتبّع البحث المسار من قصور الشبكات التقليدية وجمود الـ ASIC، إلى بنية لغة P4 ومكوّناتها وتكاملها مع مستوى التحكّم، إلى منظومة المنصات المستهدفة وأدواتها، وصولاً إلى تطبيقاتها العملية ونموذجين مكتملين بنتائج مقيسة.

ونُجيب الدراسة عن أسئلتها الثلاثة على النحو التالي. فالفقود التي فرضها ثبات مستوى البيانات — من استحالة تعريف بروتوكولاتٍ جديدة إلى دفع المنطق الشبكي قسراً نحو الخوادم — تُخفّفها P4 بمنح المطوّر القدرة التعبيرية الكاملة على معالجة الحزم. وينعكس ذلك معمارياً في إمكان تفريغ موازنة الحمل والفترة والتخزين المؤقت إلى الشبكة، وتجريب بروتوكولاتٍ مخصّصة، والحصول على قياسٍ فوريٍّ بلا أعباء. أمّا الفجوة بين أدوات المطوّر ومتطلّبات هذه التقنية فتتقلّص عبر مسارٍ تدريجيٍّ يبدأ من BMv2 المجاني وينتهي بالعتاد المتخصّص، تجسره واجهة P4Runtime التي تجعل جداول الشبكة مورداً يُدار بالشفرة المألوفة.

الدروس المستفادة للمطوّر تتلخّص في أنّ P4 ليست أداةً شبكية مخصّصة منعزلة، بل امتداداً طبيعياً لمنظومة أدواته: فمن يتقن برمجة مستوى البيانات يكسب رافعةً معمارية حقيقية في تصميم الأنظمة عالية الأداء، ويستعيد سيطرةً على طبقةٍ طالما خرجت عن يده. ومع ذلك، يبقى التنبّي محكوماً بنضج المنظومة وكلفة العتاد ومنحنى التعلّم، وهي قيودٌ على الوتيرة لا على الاتجاه.

أفق البحث المستقبلي يتّسع لتشغيل النموذجين على عتاد BMv2 و SmartNIC حقيقيٍّ لقياس إنتاجية مطلقة، ولاستكشاف تطبيقات ذات حالةٍ كرصـد DDoS، ولدمج نماذج استدلالٍ خفيفة في مستوى البيانات. وتبقى الخلاصة الجامعة أنّ الشبكة لم تعد حدّاً ثابتاً في الرسم المعماري، بل صارت — بفضل P4 — سطحاً برمجياً يتشكّل بإرادة المطوّر، وأنّ القدرة على برمجته باتت كفاءةً تنافسية في صميم هندسة الأنظمة الموزّعة الحديثة.

- [1] Bosshart, P., Daly, D., Gibb, G., Izzard, M., McKeown, N., Rexford, J., Schlesinger, C., Talayco, D., Vahdat, A., Varghese, G., and Walker, D. (2014). "P4: Programming Protocol-Independent Packet Processors." *ACM SIGCOMM Computer Communication Review*, 44(3), pp. 87–95.
- [2] McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S., and Turner, J. (2008). "OpenFlow: Enabling Innovation in Campus Networks." *ACM SIGCOMM Computer Communication Review*, 38(2), pp. 69–74.
- [3] P4 Language Consortium. (2021). "P4-16 Language Specification, Version 1.2.3." [p4.org. https://p4.org/p4-spec/docs/P4-16-v1.2.3.pdf](https://p4.org/p4-spec/docs/P4-16-v1.2.3.pdf)
- [4] Handigol, N., Heller, B., Jeyakumar, V., Lantz, B., and McKeown, N. (2012). "Reproducible Network Experiments Using Container-Based Emulation." *Proceedings of CoNEXT 2012*, ACM.
- [5] Kim, C., Sivaraman, A., Katta, N., Bas, A., Dixit, A., and Wobker, L. J. (2015). "In-band Network Telemetry (INT)." *P4 Workshop*, ACM SIGCOMM.
- [6] Testa, C., Retvari, G., and Babarczi, P. (2020). "P4-Based Traffic Engineering: A Survey." *IEEE Communications Surveys & Tutorials*, 22(4), pp. 2644–2672.
- [7] Firestone, D., Putnam, A., Mundkur, S., Chiou, D., Dabagh, A., et al. (2018). "Azure Accelerated Networking: SmartNICs in the Public Cloud." *Proceedings of USENIX NSDI 2018*, pp. 51–66.
- [8] Pontarelli, S., Bifulco, R., Bonola, M., Cascone, C., Spaziani, M., Bruschi, V., Sanvito, D., Siracusano, G., Capone, A., Honda, M., and Huici, F. (2019). "FlowBlaze: Stateful Packet Processing in Hardware." *Proceedings of USENIX NSDI 2019*.
- [9] Sultana, N., Galea, J., Greaves, D., Wójcik, M., et al. (2021). "Flightplan: Dataplane Disaggregation and Placement for P4 Programs." *Proceedings of USENIX NSDI 2021*.
- [10] Barefoot Networks / Intel. (2020). "Tofino: World's Fastest P4-Programmable Ethernet Switch ASICs." *Technical White Paper*, Intel Corporation.
- [11] Nvidia Corporation. (2022). "NVIDIA BlueField DPU Architecture." *Technical White Paper*. <https://developer.nvidia.com/networking/dpu-developer-kit>
- [12] Han, S., Marshall, S., Chun, B. G., and Ratnasamy, S. (2012). "MegaPipe: A New Programming Interface for Scalable Network I/O." *Proceedings of USENIX OSDI 2012*.
- [13] Liu, M., Cui, T., Schuh, H., Krishnamurthy, A., Peter, S., and Gupta, K. (2019). "Offloading Distributed Applications onto SmartNICs using iPipe." *Proceedings of ACM SIGCOMM 2019*, pp. 318–333.
- [14] Postel, J. (1981). "Internet Protocol — DARPA Internet Program Protocol Specification." RFC 791, IETF.
- [15] Metcalfe, R. M., and Boggs, D. R. (1976). "Ethernet: Distributed Packet Switching for Local Computer Networks." *Communications of the ACM*, 19(7), pp. 395–404.
- [16] Deering, S., and Hinden, R. (1998). "Internet Protocol, Version 6 (IPv6) Specification." RFC 2460, IETF.
- [17] p4lang Project. (2023). "behavioral-model: The reference P4 software switch (BMv2)." *GitHub*. <https://github.com/p4lang/behavioral-model>
- [18] p4lang Project. (2023). "p4c: Reference P4 Compiler." *GitHub*. <https://github.com/p4lang/p4c>
- [19] p4lang Project. (2021). "P4Runtime Specification v1.3." <https://p4.org/p4runtime/spec/v1.3.0/P4Runtime-Spec.html>

- [20] Jouet, S., and Pezaros, D. (2018). "BPFabric: Data Plane Programmability for Software Defined Networks." IEEE/IFIP NOMS 2018.
- [21] Høiland-Jørgensen, T., Brouer, J. D., Borkmann, D., Fastabend, J., Herbert, T., Ahern, D., and Miller, D. (2018). "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel." Proceedings of ACM CoNEXT 2018.
- [22] Kohler, E., Morris, R., Chen, B., Jannotti, J., and Kaashoek, M. F. (2000). "The Click Modular Router." ACM Transactions on Computer Systems, 18(3), pp. 263–297.
- [23] Kreutz, D., Ramos, F. M. V., Verissimo, P. E., Rothenberg, C. E., Azodolmolky, S., and Uhlig, S. (2015). "Software-Defined Networking: A Comprehensive Survey." Proceedings of the IEEE, 103(1), pp. 14–76.
- [24] Xilinx / AMD. (2022). "Alveo U250 Data Center Accelerator Card Product Brief." Technical Document. <https://www.xilinx.com/products/boards-and-kits/alveo/u250.html>
- [25] Ibanez, S., Brebner, G., McKeown, N., and Zilberman, N. (2019). "The P4→NetFPGA Workflow for Line-Rate Packet Processing." Proceedings of the 2019 ACM/SIGDA FPGA Symposium.

الملحق: قوائم الكود الكاملة

ملحق أ: firewall.p4 — الجدار الناري عديم الحالة

```
/* firewall.p4 - L3/L4 stateless ACL firewall for the V1Model (BMv2) architecture.
 * Author: Mohammad Khair Al-Hourani. Target: v1model / simple_switch.
 * Drops packets that match a deny-list of (srcAddr, protocol, dstPort) entries,
 * otherwise forwards by destination IPv4 longest-prefix match. */
#include <core.p4>
#include <v1model.p4>

const bit<16> TYPE_IPV4 = 0x0800;
const bit<8>  PROTO_TCP = 6;

header ethernet_t {
    bit<48> dstAddr;
    bit<48> srcAddr;
    bit<16> etherType;
}
header ipv4_t {
    bit<4>  version; bit<4> ihl;   bit<8> diffserv;
    bit<16> totalLen; bit<16> id;   bit<3> flags; bit<13> fragOffset;
    bit<8>  ttl;      bit<8> proto; bit<16> hdrChecksum;
    bit<32> srcAddr; bit<32> dstAddr;
}
header tcp_t {
    bit<16> srcPort; bit<16> dstPort;
    bit<32> seqNo;   bit<32> ackNo;
    bit<4>  dataOffset; bit<3> res; bit<9> flags;
    bit<16> window;  bit<16> checksum; bit<16> urgentPtr;
}
struct headers { ethernet_t ethernet; ipv4_t ipv4; tcp_t tcp; }
struct metadata { }

parser MyParser(packet_in pkt, out headers hdr,
                inout metadata meta, inout standard_metadata_t std) {
    state start { transition parse_ethernet; }
    state parse_ethernet { pkt.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType){
            TYPE_IPV4: parse_ipv4; default: accept; } }
    state parse_ipv4 { pkt.extract(hdr.ipv4);
        transition select(hdr.ipv4.proto){
            PROTO_TCP: parse_tcp; default: accept; } }
    state parse_tcp { pkt.extract(hdr.tcp); transition accept; }
}

control MyVerifyChecksum(inout headers hdr, inout metadata meta) { apply { } }

control MyIngress(inout headers hdr, inout metadata meta,
                  inout standard_metadata_t std) {
    action drop() { mark_to_drop(std); }
    action ipv4_forward(bit<48> dmac, bit<9> port) {
        std.egress_spec = port;
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = dmac;
        hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
    }
}
```

```

    table acl {                                // the firewall deny-list
        key = { hdr.ipv4.srcAddr: ternary;
                hdr.ipv4.proto : exact;
                hdr.tcp.dstPort : exact; }
        actions = { drop; NoAction; }
        size = 1024; default_action = NoAction();
    }
    table ipv4_lpm {                            // destination routing
        key = { hdr.ipv4.dstAddr: lpm; }
        actions = { ipv4_forward; drop; NoAction; }
        size = 1024; default_action = drop();
    }
    apply {
        if (hdr.ipv4.isValid()) {
            acl.apply();                        // deny-list first
            ipv4_lpm.apply();                  // then route survivors
        }
    }
}
control MyEgress(inout headers hdr, inout metadata meta,
                 inout standard_metadata_t std) { apply { } }
control MyComputeChecksum(inout headers hdr, inout metadata meta) { apply { } }
control MyDeparser(packet_out pkt, in headers hdr) {
    apply { pkt.emit(hdr.ethernet); pkt.emit(hdr.ipv4); pkt.emit(hdr.tcp); }
}
V1Switch(MyParser(), MyVerifyChecksum(), MyIngress(),
         MyEgress(), MyComputeChecksum(), MyDeparser()) main;

```

ملحق ب: load_balancer.p4 — موازن الحمل من المستوى الرابع

```

/* load_balancer.p4 - L4 stateless ECMP-style load balancer (V1Model / BMv2).
 * Author: Mohammad Khair Al-Hourani.
 * Hashes the 5-tuple to pick one of N backend servers so that every packet
 * of a flow lands on the same backend (per-connection consistency), while
 * different flows spread evenly. This is a "virtual IP" (VIP) front end.  */
#include <core.p4>
#include <v1model.p4>

const bit<16> TYPE_IPV4 = 0x0800;
const bit<8>  PROTO_TCP = 6;
const bit<32> VIP       = 0x0A000001; // 10.0.0.1 - the service virtual IP

header ethernet_t { bit<48> dstAddr; bit<48> srcAddr; bit<16> etherType; }
header ipv4_t {
    bit<4> version; bit<4> ihl; bit<8> diffserv; bit<16> totalLen;
    bit<16> id; bit<3> flags; bit<13> fragOffset; bit<8> ttl; bit<8> proto;
    bit<16> hdrChecksum; bit<32> srcAddr; bit<32> dstAddr;
}
header tcp_t {
    bit<16> srcPort; bit<16> dstPort; bit<32> seqNo; bit<32> ackNo;
    bit<4> dataOffset; bit<3> res; bit<9> flags; bit<16> window;
    bit<16> checksum; bit<16> urgentPtr;
}
struct headers { ethernet_t ethernet; ipv4_t ipv4; tcp_t tcp; }
struct metadata { bit<16> backend_id; }

parser MyParser(packet_in pkt, out headers hdr, inout metadata meta,
               inout standard_metadata_t std) {

```

```

state start          { transition parse_ethernet; }
state parse_ethernet { pkt.extract(hdr.ethernet);
    transition select(hdr.ethernet.etherType){ TYPE_IPV4: parse_ipv4; default: accept; } }
state parse_ipv4     { pkt.extract(hdr.ipv4);
    transition select(hdr.ipv4.proto){ PROTO_TCP: parse_tcp; default: accept; } }
state parse_tcp      { pkt.extract(hdr.tcp); transition accept; }
}
control MyVerifyChecksum(inout headers hdr, inout metadata meta) { apply { } }

control MyIngress(inout headers hdr, inout metadata meta,
    inout standard_metadata_t std) {
    action drop() { mark_to_drop(std); }
    action set_backend(bit<32> dip, bit<48> dmac, bit<9> port) {
        hdr.ipv4.dstAddr    = dip;           // DNAT to the chosen backend
        hdr.ethernet.dstAddr = dmac;
        std.egress_spec     = port;
        hdr.ipv4.ttl        = hdr.ipv4.ttl - 1;
    }
    table backend_pool {
        key = { meta.backend_id: exact; }
        actions = { set_backend; drop; }
        size = 256; default_action = drop();
    }
    apply {
        if (hdr.ipv4.isValid() && hdr.ipv4.dstAddr == VIP) {
            // hash the 5-tuple -> stable backend index in [0, 3]
            hash(meta.backend_id, HashAlgorithm.crc16,
                (bit<16>)0,
                { hdr.ipv4.srcAddr, hdr.ipv4.dstAddr,
                  hdr.ipv4.proto, hdr.tcp.srcPort, hdr.tcp.dstPort },
                (bit<16>)4);
            backend_pool.apply();
        }
    }
}
control MyEgress(inout headers hdr, inout metadata meta,
    inout standard_metadata_t std) { apply { } }
control MyComputeChecksum(inout headers hdr, inout metadata meta) { apply { } }
control MyDeparser(packet_out pkt, in headers hdr) {
    apply { pkt.emit(hdr.ethernet); pkt.emit(hdr.ipv4); pkt.emit(hdr.tcp); }
}
V1Switch(MyParser(), MyVerifyChecksum(), MyIngress(),
    MyEgress(), MyComputeChecksum(), MyDeparser()) main;

```

ملحق ج: ملاحظة حول إعادة الإنتاج

حُفِظَت مَلَفَات البرنامجين وشيفرة التجربة (sim.py و microbench.py) وملف النتائج (RESULTS.json) ضمن مجلد prototype المرافق للبحث، بما يُتيح إعادة تشغيل التجربة والتحقق من كل رقم ورد في الفصل الرابع.

<https://github.com/Khair69/P4-researchpaper>